



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Sistemas de la información

**Diseño de un sistema de telemetría para el análisis,
visualización y ayuda a la toma de decisiones
aplicado al deporte de motor y alta competición**

**Design of a telemetry system for analysis,
visualization and decision support applied to
high-performance motorsport**

Realizado por

Darío Alessandro Di Totaro Ruíz

Tutorizado por

Francisco Javier Ferrer Urbano

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, Mayo, 2026

Resumen

Este trabajo centra sus esfuerzos en el desarrollo de una herramienta gratuita de código abierto que permita a los usuarios analizar los múltiples aspectos fundamentales que influyen en el deporte de motor. Dada la amplitud de las diferentes categorías dentro de esta disciplina nos centraremos en la categoría más influyente, la Fórmula 1. Mediante un *dashboard*, el usuario obtiene una visualización detallada de los pilotos y escuderías presentes en el transcurso de un gran premio, facilitando un análisis sencillo de cada carrera.

El principal objetivo de este proyecto es proporcionar una herramienta intuitiva para personas con un conocimiento de base del deporte, personas con un interés por el *simracing* o usuarios que quieran introducirse en la modalidad de una forma más técnica. Los usuarios pueden interactuar con los diferentes apartados y analizar detalles que puedan ayudar a mejorar su conocimiento sobre el deporte o comportamiento de los monoplazas analizando resultados, ritmo de carrera, distintas estrategias, fuerzas G del trazado o estrés del neumático. El desarrollo se ha llevado a cabo utilizando Streamlit, un *framework* de Python de código abierto que permite construir aplicaciones web para el análisis de datos con capacidad para integrar gráficos. Python se ha utilizado como lenguaje aprovechando bibliotecas como NumPy, Pandas o Plotly para los distintos procesos que conlleva el manejo de datos. Se obtienen datos del monoplaza mediante el uso de APIs públicas como Ergast y la biblioteca FastF1 para la recolección de datos de la API de la Fórmula 1 así como una caché local para mejorar los tiempos de carga.

Se ha identificado como área de posible expansión la integración de datos de otras competiciones con mayor accesibilidad de datos, como el WEC (*World Endurance Championship*) o categorías de Fórmula inferiores. El resultado es una herramienta de análisis post-sesión que integra en una interfaz la visualización de telemetría, estrategia de carrera y clasificación automática de estilos de pilotaje.

Palabras clave: Fórmula 1, Análisis de datos, Python, Telemetría, Streamlit, Dashboard, Deportes

Abstract

This work centers its efforts on the development of a free open-source tool that allows users to analyze the multiple fundamental aspects that influence motorsport. Given the breadth of the different categories within this discipline, we will focus on the most influential category, Formula 1. Through a dashboard the user obtains a detailed visualization of the drivers and teams present during the course of a grand prix, facilitating a simple analysis of each race.

The main objective of this project is to provide an intuitive tool for people with a base knowledge of the sport, people with an interest in simracing or users who want to introduce themselves to the modality in a more technical way. Users can interact with the different sections and analyze details that can help to improve their knowledge about the sport or behavior of the single-seaters analyzing results, race pace, different strategies, G-forces of the track or tire stress. The development has been carried out based on Streamlit, an open-source Python framework that allows building web applications for data analysis with the capacity to integrate graphics. Python has been used as the language taking advantage of libraries like NumPy, Pandas or Plotly for the different processes that data handling entails. Data of the car is obtained through the use of public APIs like Ergast and the FastF1 library for data collection from the Formula 1 API as well as a local cache to improve loading times.

It has been identified as an area of possible expansion the integration of data from other competitions with greater data accessibility, such as the WEC (World Endurance Championship) or lower Formula categories. The result is a post-session analysis tool that integrates telemetry visualization, race strategy and automatic classification of driving styles into a single interface, bringing to the average fan an analysis previously reserved for the teams.

Keywords: Formula 1, Data analysis, Python, Telemetry, Streamlit, Dashboard, Sports

Agradecimientos

A mi familia, lo primero y lo más importante. Por estar siempre, por el apoyo incondicional en cada paso de estos años y por no dejarme tirar la toalla cuando tocaba. Nada de esto habría sido posible sin vosotros, y este trabajo es tan vuestro como mío.

A mis amigos y compañeros de carrera, por las horas en la biblioteca y las que no fueron en la biblioteca, por hacer que estos años hayan merecido la pena.

A mi tutor, Francisco Javier Ferrer Urbano, por su orientación y seguimiento constante a lo largo del desarrollo de este trabajo.

Y, en general, a todos los que de una forma u otra han caminado conmigo hasta aquí. Gracias.

Índice General

Resumen	1
Abstract	3
Agradecimientos	5
Índice General	7
Índice de Figuras	11
Índice de Tablas	15
Índice de Listados	17
1 Introducción y Objetivos	19
1.1 Introducción	19
1.2 Motivación	20
1.3 Objetivos	21
1.4 Estructura de la memoria	23
1.5 Uso de IA	23
2 Marco Teórico y Estado del arte	25
2.1 Marco Contextual Teórico	25
2.1.1 La Fórmula 1 y su estructura	25
2.1.2 El fin de semana de un Gran Premio	26
2.1.3 Desarrollo de una carrera	26
2.1.4 Los compuestos de neumático	26
2.1.5 Telemetría	27
2.1.6 Comparación y sincronización	27
2.2 Estado del arte	28
2.2.1 Herramientas profesionales	28
2.2.2 Ecosistema de código abierto	30
2.2.3 Proyectos y Herramientas de la comunidad	31
2.2.4 Investigación académica	33
3 Metodología, Decisiones de análisis y diseño del sistema	37
3.1 Metodología	37

3.1.1	Modelo de desarrollo	37
3.1.2	Seguimiento con el tutor	38
3.1.3	Gestión de tareas	38
3.1.4	Control de versiones	38
3.1.5	Planificación temporal	39
3.2	Análisis de requisitos	39
3.3	Tecnologías empleadas	42
3.4	Diseño del sistema	42
3.4.1	FastF1 sobre otras alternativas	44
3.4.2	Streamlit para interfaz	44
3.4.3	Estado de la sesión	45
3.4.4	Caché de dos niveles	45
3.4.5	Módulos independientes entre sí	45
3.4.6	K-Means pre-entrenado	45
3.4.7	Interfaz Visual	46
4	Implementación	47
4.1	Estructura del código	47
4.2	Modelo y Acceso a datos	50
4.2.1	Origen y naturaleza de datos	51
4.2.2	Modelo Jerárquico de datos	51
4.2.3	Patrón de preparación	52
4.2.4	Caché	53
4.3	Información de sesión	55
4.3.1	Resumen de la sesión	55
4.3.2	Cronología de Eventos	56
4.3.3	Estrategia de neumáticos	58
4.4	Análisis de piloto	59
4.4.1	Resumen del Piloto	60
4.4.2	Mapas sobre el trazado: velocidad y marchas	61
4.4.3	Telemetry Trace	62
4.5	Análisis comparativo	64
4.5.1	Panel de Equipo	65

4.5.2	Comparativa de sesión	66
4.5.3	La vuelta ideal	66
4.5.4	Coche fantasma	67
4.6	Dinámica vehicular	68
4.6.1	Fundamento físico	68
4.6.2	El Diagrama G-G	71
4.6.3	Carga Aerodinámica	71
4.6.4	Carga por rueda	72
4.7	Modelos de aprendizaje no supervisado	73
4.7.1	El problema y el enfoque no supervisado	73
4.7.2	Representación de las características	73
4.7.3	Cómo agrupa K-Means	75
4.7.4	Refinamiento sobre el modelo base	75
5	Validación y Pruebas	77
5.1	Validación funcional	77
5.2	Pruebas Unitarias	78
5.2.1	Estrategia: telemetría sintética frente a <i>mocking</i>	78
5.2.2	Niveles de verificación	79
5.3	Validación de los clasificadores	80
5.3.1	Métricas empleadas	80
5.3.2	Selección del número de clústeres	80
5.3.3	Resultados	83
6	Conclusiones y Trabajos futuros	85
6.1	Trabajo Futuro	86
	Apéndice A. Manual de Instalación	87
A.1	Requisitos previos	87
A.2	Obtención del código fuente	87
A.3	Instalación en Windows	87
A.4	Instalación en Linux	88
A.5	Primera ejecución	89
	Apéndice B. Manual de Usuario	91
B.1	Buscador de Gran Premio	91

B.2	Selección de sesión	92
B.3	Hub de módulos	92
B.4	Módulos de análisis	92
B.4.1	Información de Sesión	93
B.4.2	Análisis de Piloto	93
B.4.3	Comparativas	94
B.4.4	Dinámica Vehicular	96
B.5	Selección de piloto y vuelta	96

Índice de Figuras

2.1	Sincronización de dos vueltas por interpolación sobre una rejilla de distancia común. Las muestras originales de cada vuelta, registradas en posiciones distintas, se re-muestran sobre puntos equiespaciados compartidos que permiten compararlas directamente.	27
2.2	Ejemplo de gráfico <i>Battle Forecast</i> de AWS F1 Insights emitido durante una carrera. .	29
2.3	Interfaz de F1Dash, dashboard de telemetría en tiempo real basado en FastF1.	32
3.1	Ejemplo de gráfico <i>GitHub Projects</i> durante la realización del trabajo.	38
3.2	Tecnologías utilizadas en el desarrollo de TALOS.	42
3.3	Diagrama de navegación y arquitectura de vistas de TALOS.	43
3.4	Las tres vistas secuenciales de la navegación de TALOS: el mapa mundial de Grandes Premios (<i>Grid</i>), la selección de sesión (<i>Sessions</i>) y el hub de módulos de análisis (<i>Analysis</i>).	44
4.1	Vista de Resumen de la sesión: mapa del circuito dibujado desde la telemetría, condiciones meteorológicas y estadísticas de la carrera.	56
4.2	Vista de Cronología de Eventos: reconstrucción de la secuencia de banderas, <i>Safety Car</i> y <i>VSC</i> a lo largo de la sesión a partir de los códigos de <code>track_status</code>	57
4.3	Accidente de Russell, GP de Australia 2024	58
4.4	Vista de Estrategia de Neumáticos: diagrama de Gantt de <i>stints</i> por piloto, reparto de compuestos y curva de degradación.	59
4.5	Vista de Resumen del Piloto: KPIs de la vuelta, tiempos por sector y distribución de la sesión.	60
4.6	Mapa de velocidad: el trazado del circuito coloreado según la velocidad en cada punto. El mapa de marchas es análogo, codificando la marcha seleccionada.	61
4.7	Efecto del DRS sobre el flujo de aire	62
4.8	Vista de Telemetry Trace: evolución de los canales de telemetría (velocidad, acelerador, freno, marcha, RPM y DRS) a lo largo de la vuelta frente a la distancia.	63

4.9	Los dos clasificadores en acción sobre una vuelta de Daniel Ricciardo (GP de Australia 2024): arriba el tipo de vuelta (<i>Lift & Coast</i> , gestión) y abajo la firma de pilotaje (<i>Reactivo</i>), ambos calculados en vivo. Su construcción se detalla en la Sección 4.7.	63
4.10	Simulación animada de la vuelta: un punto recorre el trazado mientras se desplazan los paneles de pedales y se marcan zonas de DRS y tiempos por sector.	64
4.11	La Vuelta Ideal: el circuito dividido en microsectores, cada uno coloreado con el equipo más rápido en él y filtrado por equipos. El tiempo resultante es la suma de los mejores parciales de toda la parrilla.	67
4.12	Coche Fantasma (Qualy GP de Mónaco 2023): dos vueltas interpoladas sobre la rejilla común se reproducen como persecución; abajo, la curva del delta acumulado entre ambas.	68
4.13	Visualización CFD ilustrativa del flujo aerodinámico	72
4.14	Mapa de Carga por Rueda: estimación de la carga vertical y la exigencia sobre cada neumático a lo largo de la vuelta, derivada de las transferencias de masa. Curva número 1 de brasil, frenada fuerte.	74
5.1	Selección de k para el clasificador de tipo de vuelta: inercia (codo) y coeficiente de Silhouette frente al número de clústeres. El máximo del Silhouette se alcanza en $k = 4$	81
5.2	Selección de k para el clasificador de estilo de pilotaje. En $k = 4$ el silhouette es de 0,45, a solo 0,01 del máximo, pero con grupos interpretables y sin fragmentar en exceso a los pilotos.	82
5.3	Selección de k para el clasificador de circuitos. Se adopta $k = 5$ por la interpretabilidad de las cinco tipologías resultantes, pese a que un k menor ofrezca un silhouette superior.	82
5.4	Clústeres de circuitos proyectados sobre sus dos componentes principales (95,8% de la varianza). Mónaco queda completamente aislado del resto, confirmando su carácter de trazado único.	84
5.5	Clústeres de estilo de pilotaje proyectados sobre sus dos componentes principales (64,5% de la varianza), que sustentan la separación de los cuatro perfiles.	84
6.1	Buscador de Gran Premio: mapamundi interactivo del calendario.	91
6.2	Pantalla de selección de sesión del fin de semana.	92
6.3	Hub de módulos con la sesión activa, el podio y el panel de métricas.	93

6.4	Módulo <i>Panel de Equipo</i> : comparación lado a lado de los dos pilotos de una escudería (telemetría, ritmo, neumáticos y circuito animado).	95
-----	--	----

Índice de Tablas

2.1	Comparativa de herramientas de análisis de telemetría de Fórmula 1.	35
3.1	Temporización del proyecto por fases.	39
3.2	Requisitos funcionales del sistema.	39
3.3	Requisitos no funcionales del sistema.	41
3.4	Organización de módulos de TALOS por categoría.	43
5.1	Casos de validación funcional, uno por categoría de la herramienta, contrastados contra fuentes oficiales y conocimiento del dominio.	78
5.2	Cobertura de las pruebas unitarias por componente.	79
5.3	Métricas de validación de los tres clasificadores. ARI obtenido por validación cruzada (<i>leave-one-out</i> para circuito y pilotaje, dado su menor tamaño, y <i>5-fold</i> para tipo de vuelta).	83

Índice de Listados

4.1	Estructura de directorios del proyecto.	48
4.2	Registro declarativo de la navegación en <code>app.py</code>	48
4.3	Campos representativos de cada nivel de datos de FastF1.	51
4.4	Patrón de acceso a telemetría en <code>Telemetrytrace.py</code>	52
4.5	Gestión de estado y caché en <code>session_loader.py</code>	53
4.6	Interpolación de una vuelta sobre la rejilla común en <code>Ghostcar.py</code>	65
4.7	Cálculo de las fuerzas G en <code>GGDiagram.py</code>	70
6.1	Clonado del repositorio	87
6.2	Instalación en Linux	88
6.3	Arranque en Linux	88

1

Introducción y Objetivos

1.1 Introducción

La Fórmula 1 es la categoría con más prestigio del automovilismo deportivo y uno de los entornos tecnológicos más exigentes de todo el mundo. Cada coche incorpora cerca de 300 sensores que generan 2 GB de datos por vuelta de forma aproximada. Estos datos abarcan desde la temperatura individual de cada neumático hasta la presión del aire y la carga aerodinámica sobre puntos concretos de la carrocería. Esta vasta cantidad de información construye la base sobre la que los equipos terminarán tomando decisiones estratégicas en tiempo real: cuándo realizar una parada en boxes, cómo ajustar el balance de carga en el monoplaza o cómo adaptar el estilo de conducción según el estado de la pista y del propio vehículo. Estos datos no solo influyen para las carreras en sí sino a lo largo de toda la temporada para poder desarrollar el coche aplicando mejoras a las piezas de la carrocería.

La relación entre la Fórmula 1 y los datos no es reciente. Ya en la década de 1980, equipos como McLaren comenzaron a introducir sistemas rudimentarios de telemetría transmitiendo parámetros básicos del motor al garaje. Con la llegada de los sistemas de adquisición de datos digitales la información es mucho más accesible multiplicando los datos disponibles convirtiendo a la figura del ingeniero en una figura con casi tanta importancia como la del propio piloto. Hoy, un fin de semana genera varios terabytes de información que es analizada en tiempo real desde los *command centers* de las fábricas en Milton Keynes, Maranello o Brackley, a miles de kilómetros del circuito. La telemetría dejó de ser una herramienta de diagnóstico para convertirse en el pilar central de la toma de decisiones

en la categoría más exigente del automovilismo.

Sin embargo la mayor parte de esta información es altamente confidencial ya que las escuderías la consideran como secretos empresariales e información competitiva. Los datos que se publican de forma oficial son un subconjunto altamente reducido como los tiempos de vuelta, compuestos del neumático, posición GPS y telemetría básica del piloto. La librería de código abierto FastF1 permite acceder a estos datos históricos de una forma estructurada y ha impulsado en los últimos años una comunidad creciente de análisis aficionado.

A pesar de la disponibilidad de estos datos, las herramientas existentes para su explotación presentan limitaciones importantes: Son fragmentarias, cada *script* analiza un solo aspecto de forma aislada, requieren conocimientos de programación para su uso y no ofrecen una experiencia integrada que permita navegar entre los distintos tipos de análisis sobre una misma sesión. Por otro lado las opciones propietarias, que son las que las propias escuderías utilizan, tienen un elevado coste de licencia por lo que no es una opción recomendada para el aficionado promedio. Los usuarios carecen de una plataforma unificada y gratuita que les permita explorar el máximo de telemetría posible con profundidad e interactividad que los datos permiten.

Telemetry Analysis and Lap Optimization System (TALOS de ahora en adelante) nace para cubrir este hueco. Se trata de un sistema de soporte a la decisión (DSS) orientado al análisis interactivo de telemetría histórica de Fórmula 1, construido sobre datos públicos y herramientas de código abierto. Es decir, TALOS opera sobre datos ya registrados realizando un análisis **post-sesión** o asíncrono, una vez ha concluido la sesión. No se trata por tanto de un sistema de telemetría a tiempo real como los que emplean los equipos durante la competición, sino de una herramienta de estudio y exploración a posteriori.

1.2 Motivación

El interés técnico de este trabajo reside en el conjunto de varias disciplinas de la Ingeniería informática aplicadas a un dominio de datos real. La telemetría obtenida no es un conjunto de datos completamente limpio: presenta muestreo irregular, valores ausentes y muestras irregulares además que muestra una frecuencia mucho más baja de datos que los sensores que usan las escuderías. Procesarla correctamente exige técnicas de filtrado, en particular el filtro de Savitzky-Golay [27] para preservar derivadas mientras se elimina el ruido de alta frecuencia.

Sobre esta base se construyen 2 motores de inferencia. El primero es un clasificador de circuitos basado en reglas sobre características extraídas de un conjunto extenso de vueltas, esto categoriza el trazado en una de cinco tipologías. El segundo es un clasificador de estilos de conducción sobre métricas de comportamiento de piloto. Ambos clasificadores están basados en *K-Means* no supervisado.

Desde el punto de vista funcional, la motivación principal es hacer accesible un tipo de análisis reservado a ingenieros de escudería y ponerlo al alcance del aficionado. Sirve como punto de entrada al aficionado promedio que desee entender cómo funciona el deporte con una mayor profundidad e incluso puede atraer a gente nueva al deporte, explicando cómo funciona todo de una manera detallada y que una persona con poco conocimiento sobre el dominio del deporte pueda entender. TALOS no compite con las herramientas profesionales, sino que traslada su filosofía analítica vista desde un punto gratuito e interactivo.

El origen personal de este trabajo es anterior a la ingeniería. La Fórmula 1 fue durante años un ritual familiar de los domingos habiendo incluso estado en circuitos de forma presencial. Este interés se retomó en la etapa universitaria con una nueva dimensión, la del *simracing*, donde mejorar los tiempos de vuelta exige exactamente el mismo tipo de análisis que realizan los ingenieros de pista aunque con datos mucho más limitados. Fue esa mentalidad de optimización y entender el dato la que orientó la elección del tema. La propuesta inicial era más ambiciosa, un sistema de predicción de estrategia de carreras. Sin embargo durante las fases tempranas quedó patente que la validación de dichas predicciones era inviable sin acceso a la información que las escuderías no publican. El proyecto pivotó hacia un objetivo más sólido y validable: construir una herramienta de análisis e interpretación de la telemetría existente, que es precisamente lo que TALOS es.

1.3 Objetivos

La Fórmula 1 es hoy en día un deporte difícil de entender por primera vez. No porque sea aburrido, sino porque ocurren demasiadas cosas a la vez: estrategias de neumáticos en vueltas tempranas, adelantamientos que se “cocinan” con más de tres vueltas de antelación, diferencias de rendimiento entre pilotos del mismo equipo. El aficionado nuevo se encuentra con un deporte que no solo exige ver las carreras, sino que exige contexto antes de poder disfrutarlo, y ese contexto no está siempre disponible de forma accesible.

Este trabajo nace con la intención de ser este contexto. No un sustituto de la carrera sino una

herramienta que permita entenderla mejor, antes o después de verla. Que un usuario pueda abrir la aplicación tras el gran premio y entender visualmente y analíticamente por qué un piloto fue el más veloz en un sector, o qué diferencia hay entre el punto de frenada con otro piloto, como acabó la carrera, clasificación o entrenamientos. Alguien que lleva siguiendo el deporte desde hace años pueda encontrar en los datos alguna confirmación o una contradicción de lo que creía saber.

Hay también un público al que no se suele mencionar por pertenecer a un nicho de espectadores que hoy en día se está volviendo más grande y representa quizás el caso de uso más interesante: el jugador de *simracing*. Quien practica simulación de carreras ya tiene una intuición sobre lo que significa perder tiempo en una curva o gestionar los neumáticos, pero rara vez tiene acceso a datos reales con los que calibrar esa intuición. Esta herramienta puede ser para ese usuario lo que un ingeniero de datos sería en un equipo profesional. Nótese que la herramienta está diseñada de forma modular, por lo que es posible añadir una nueva funcionalidad para ver los propios datos de telemetría capturando paquetes UDP de videojuegos de Fórmula 1 o simuladores como Assetto Corsa.

Existe además un tercer perfil al que pocas herramientas prestan atención: el que llega al deporte sin bagaje previo. Alguien que empieza a ver carreras y no entiende por qué un coche que va primero puede acabar tercero, o qué significa que un piloto *gestione* los neumáticos. Para ese perfil la herramienta puede funcionar como punto de entrada: no como un tutorial sino como un espejo en el que ver reflejado lo que ha ocurrido con datos reales detrás. Ver la telemetría no requiere saber de ingeniería, requiere de curiosidad, y eso es algo que la herramienta no puede ni necesita exigir. En ese sentido, el proyecto no tiene un tipo único de usuario sino un espectro. En un extremo, el aficionado ocasional que quiere entender por qué su piloto favorito perdió la carrera, en el otro, el entusiasta técnico que quiere comparar las fuerzas G de dos trazadas distintas. TALOS está diseñado para ser útil en ambos extremos sin resultar intimidante en ninguno de los dos.

Lo que une todos estos casos de uso es la misma convicción: que los datos de la Fórmula 1 ya son públicos y accesibles, y que lo que faltaba no era más información sino una forma mejor de explorarla. Ese es el hueco que este trabajo intenta cubrir.

En síntesis, los objetivos específicos del proyecto pueden concretarse en cuatro:

- **Democratizar el análisis de telemetría**, acercando al aficionado un tipo de análisis tradicionalmente reservado a los equipos.
- **Diseñar una experiencia de usuario adaptable** a un espectro amplio de perfiles, del aficionado ocasional al entusiasta técnico o al jugador de *simracing*.

- **Garantizar la modularidad y extensibilidad del sistema**, de modo que puedan incorporarse nuevos análisis o fuentes de datos sin alterar el núcleo.
- **Aportar valor analítico en el análisis post-sesión**, integrando en una sola herramienta la visualización de telemetría, la estrategia y la clasificación automática.

1.4 Estructura de la memoria

El presente documento se organiza en seis capítulos que siguen el desarrollo natural del proyecto, desde el contexto que lo motiva hasta las conclusiones que se extraen de su realización.

- El **capítulo 2** sitúa la herramienta dentro del panorama de análisis de telemetría. Describe el entorno actual y herramientas similares.
- El **capítulo 3** recoge las decisiones de análisis y diseño del sistema: arquitectura, organización en categorías y criterios que guiaron las elecciones más relevantes.
- El **capítulo 4** detalla la implementación: la capa de integración con FastF1, el procesamiento de telemetría, la visualización y los motores de inferencia.
- El **capítulo 5** presenta la validación del sistema, pruebas y criterios utilizados para comprobar que los resultados son correctos y reproducibles.
- El **capítulo 6** cierra el trabajo con las conclusiones, valoración del cumplimiento de objetivos y las líneas de trabajo futuro identificadas.
- Los **anexos** recogen el manual de usuario, la documentación técnica y material complementario que apoya el contenido de los capítulos anteriores.

1.5 Uso de IA

Durante el transcurso de este trabajo se han usado herramientas de inteligencia artificial generativa, concretamente asistentes de código basados en modelos de lenguaje como Gemini o Claude, como apoyo en la implementación del sistema. Su uso ha abarcado la exploración de alternativas de implementación

para los distintos módulos de TALOS así como decisiones de diseño, imágenes como *placeholders* o fragmentos de código.

El uso de estas herramientas es comparable al de cualquier otro recurso de apoyo al desarrollo: del mismo modo que se consulta la documentación, foros o ejemplos externos, los asistentes de IA han actuado como interlocutores técnicos en el proceso de construcción del sistema. Las decisiones de diseño, definición de funcionalidades, elección de tecnologías, criterios de validación y análisis y supervisión de los resultados han sido responsabilidad del autor.

El uso de IA puede ser una útil herramienta siempre que se use de manera responsable y ética.

2

Marco Teórico y Estado del arte

El análisis de telemetría en competición ha seguido históricamente dos caminos paralelos: el de las herramientas profesionales, propietarias y de acceso restringido a los equipos, y el ecosistema abierto construido por la comunidad aprovechando los datos que las organizaciones publican. Este capítulo recorre ambos caminos, sitúa las iniciativas académicas y abiertas más relevantes y concluye con un análisis comparativo que justifica el hueco de este proyecto. Además, sitúa el marco teórico del dominio aclarando conceptos clave que serán útiles para entender el desarrollo de este proyecto.

2.1 Marco Contextual Teórico

Este trabajo se sitúa en la intersección entre el análisis de datos y un dominio muy específico, el de la Fórmula 1, cuyos conceptos no resultan evidentes para un lector con formación técnica general. Esta sección introduce las nociones mínimas, tanto del deporte como del tipo de datos que maneja, necesarias para comprender el resto de la memoria.

2.1.1 La Fórmula 1 y su estructura

La Fórmula 1 es la máxima categoría del automovilismo. Se disputa como un campeonato mundial que, a lo largo de un año, recorre del orden de veinticuatro **Grandes Premios** repartidos por distintos países. En la parrilla compiten actualmente once (Introducido el undécimo equipo en la temporada de

2026) escuderías o equipos, cada una con dos pilotos, lo que suma veintidós pilotos. De su rendimiento dependen dos clasificaciones paralelas, el campeonato de pilotos y el de constructores, que premia a los equipos.

2.1.2 El fin de semana de un Gran Premio

Un GP no es un único evento, sino un conjunto de **sesiones** repartidas a lo largo de un fin de semana, distinción fundamental para este proyecto ya que la herramienta organiza todo su análisis en torno a esta estructura. Las sesiones de **práctica** (libres) sirven para poner a punto el coche. La **clasificación** determina el orden de salida buscando la vuelta más rápida posible. Finalmente, la **carrera** es el evento principal. Algunos GP incluyen además una sesión de **sprint** siendo una versión más corta de una carrera.

2.1.3 Desarrollo de una carrera

Durante la carrera, los monoplazas completan un número fijo de vueltas al circuito partiendo desde una parrilla ordenada por la clasificación. A lo largo de la prueba, los coches realizan **paradas en boxes** para cambiar los neumáticos, lo que divide la carrera de cada piloto en tramos llamados **stints**, cada uno asociado a un juego de neumáticos. Las posiciones se disputan mediante adelantamientos en pista y, de forma menos evidente, mediante la estrategia de cuándo y cómo realizar esas paradas.

2.1.4 Los compuestos de neumático

El neumático es probablemente el elemento más determinante del rendimiento en Fórmula 1, y su gestión condiciona toda la estrategia de carrera. El proveedor único suministra varios **compuestos** de neumático seco, identificados por colores que van desde el **blando** (*soft*) hasta el **duro** (*hard*), pasando por el **medio** (*medium*). La diferencia entre ellos es fundamental puesto que un compuesto más blando ofrece un mayor agarre y por tanto da vueltas más rápidas, pero se desgasta antes. Un compuesto más duro es más lento pero aguanta muchas más vueltas. A esto se añaden los neumáticos de lluvia, intermedios y extremos para pista mojada. A medida que un neumático acumula kilómetros pierde rendimiento de forma progresiva, este fenómeno es conocido como **degradación**, que se traduce en tiempos por vuelta cada vez mayores. Elegir qué compuesto montar en cada momento es una de las

decisiones estratégicas más importantes de una carrera.

2.1.5 Telemetría

Se denomina **telemetría** al conjunto de datos que el coche registra sobre su propio estado durante la conducción. Cada monoplaza incorpora cientos de sensores que miden variables de forma continua. Estos datos se capturan a una **frecuencia de muestreo** determinada, es decir, un número fijo de muestras por segundo (en los datos públicos en torno a cuatro por segundo). Conocer esta frecuencia es imprescindible para reconstruir correctamente magnitudes, como aceleraciones o posición.

2.1.6 Comparación y sincronización

Una de las operaciones más habituales en el análisis es comparar dos vueltas, ya sean de pilotos o de la misma vuelta en momentos diferentes. El problema es que cada vuelta se registra de forma independiente, por lo que sus muestras no coinciden en los mismos puntos del trazado. Es necesario pues, para que tenga sentido, un proceso de **sincronización**, que consiste en alinear ambas series sobre una referencia común, normalmente la distancia recorrida. La Figura 2.1 ilustra este proceso, las muestras originales de cada vuelta (en distintas posiciones) se remuestrean sobre una rejilla de distancia compartida que las hace comparables punto a punto.

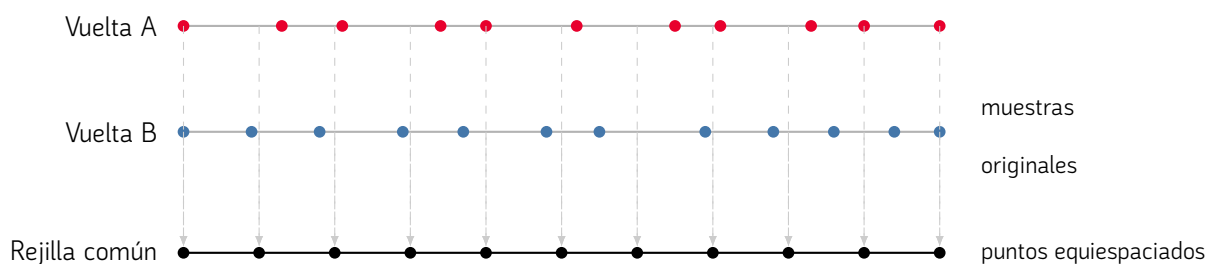


Figura 2.1. Sincronización de dos vueltas por interpolación sobre una rejilla de distancia común. Las muestras originales de cada vuelta, registradas en posiciones distintas, se remuestrean sobre puntos equiespaciados compartidos que permiten compararlas directamente.

2.2 Estado del arte

El análisis de telemetría en competición ha seguido históricamente dos caminos paralelos, el de las herramientas profesionales, propietarias y de acceso restringido a los equipos, y el del ecosistema abierto construido por la comunidad a partir de los datos que las organizaciones publican. Esta sección recorre ambos, sitúa las iniciativas académicas más relevantes y concluye con un análisis comparativo que justifica el hueco que viene a cubrir este proyecto.

2.2.1 Herramientas profesionales

El primer camino lo ocupan las herramientas profesionales que los propios equipos y organizadores emplean. Son soluciones potentes pero propietarias, de coste elevado y acceso restringido, lo que las sitúa fuera del alcance del aficionado. Esta sección repasa las más representativas.

2.2.1.1 McLaren Applied ATLAS

McLaren Applied es el proveedor exclusivo de la unidad de control electrónico estándar (SECU por sus siglas en inglés) de todos los equipos de Fórmula 1 desde 2008, contrato que renovó hasta 2030. Esta posición convierte su plataforma de análisis ATLAS (*Advanced Telemetry Linked Acquisition System*), en el estándar de *facto* del *paddock*, todos los monoplazas generan datos a través del mismo hardware, lo que facilita la adopción de un entorno común [17].

ATLAS captura, distribuye y visualiza datos procedentes de los sistemas de control del monoplaza. La telemetría opera en modo “ráfaga” enviando así todos los datos almacenados de una vuelta al pasar por un punto concreto. De igual manera incorpora modelos de aprendizaje para predecir degradación del neumático, consumo de combustible y eventos de carrera a medida que avanza la sesión. En 2023 McLaren Applied integró la base de datos de series temporales KX para añadir capacidades de consulta analítica sobre datos de telemetría en tiempo real.

ATLAS es el software comercial cerrado, disponible únicamente bajo contrato con McLaren Applied. No existe versión libre ni documentación pública de sus interfaces. Su coste y sus requisitos de infraestructura lo sitúan fuera del alcance del aficionado promedio.

2.2.1.2 AWS F1 Insights

Desde 2018 Amazon Web Services es el socio oficial de nube e inteligencia artificial de la Fórmula 1 [2]. La colaboración se materializa en dos planos.

El plano más visible son las **F1 Insights**: infografías generadas por IA que aparecen durante las retransmisiones televisivas. El catálogo supera los veinticuatro indicadores, entre los que se encuentran *Battle Forecast* (cuántas vueltas hasta que alcanza la distancia de adelantamiento), *Undercut Threat* (análisis de la estrategia de parada en boxes), *Tyre Performance* (estimación de ángulos de deslizamiento y energía de desgaste), *Corner Analysis* (desglose de la curva en 4 fases) o *Driver Performance* (evaluación del piloto sobre siete métricas). En 2025 se añadió *Track Pulse*, basado en IA generativa (Amazon Bedrock), que sintetiza la situación a tiempo real en pista para los equipos de producción televisiva. La Figura 2.2 muestra un ejemplo del indicador *Battle Forecast* emitido durante una retransmisión.

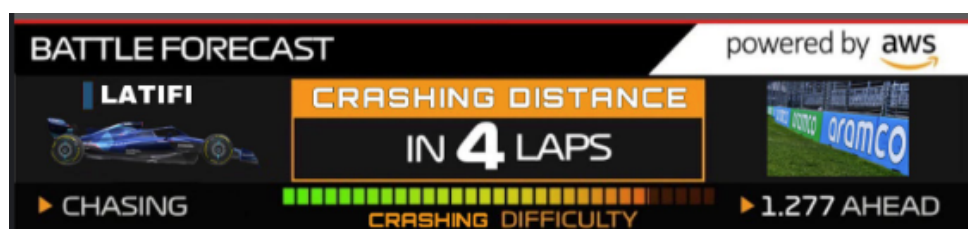


Figura 2.2. Ejemplo de gráfico *Battle Forecast* de AWS F1 Insights emitido durante una carrera.

Sin embargo estas herramientas no son un sistema de análisis para los equipos sino un producto de entretenimiento para el espectador. Cada equipo recibe únicamente los datos de su propio coche a través de canales propietarios y AWS procesa los datos agregados de los monoplazas con fines de difusión. Existe una arquitectura llamada Amazon S3 que cuenta con décadas de datos históricos, redes neuronales y *pipelines* de ML. Esta arquitectura es de gran interés técnico, pero no expone ninguna interfaz accesible al exterior.

2.2.1.3 Otras plataformas comerciales: MoTeC y Bosch Motorsport

MoTeC es una empresa australiana con amplia presencia en categorías inferiores a la F1 (Fórmula 2, Fórmula 3, resistencia, turismos). Su software de análisis i2 permite superposición de vueltas, histogramas, análisis de presión de freno y comparación de aceleración en base tiempo y distancia. La versión estándar se incluye con la compra de hardware MoTeC (Registradores como el C70 o C80, cuyo coste asciende a varios miles de euros). La versión Pro requiere licencia adicional. En la Fórmula 1 MoTeC no compete directamente con ATLAS dada la relación contractual de McLaren Applied con la FIA, pero la filosofía analítica es un referente conceptual en el campo.

Bosch Motorsport ofrece registradores de datos profesionales con software de análisis propio. El acceso está en ambos casos vinculado a la adquisición de hardware profesional.

2.2.1.4 Datos oficiales de la Fórmula 1

La Fórmula 1 publica datos de la sesión a través de una API de sincronización en directo que no requiere autenticación, no hay documentación oficial de dicha API. El contenido disponible incluye tiempos de vuelta, sectores, compuesto del neumático, posición GPS simplificada y telemetría básica del piloto. Todos estos datos son propiedad de Formula One World Championship Limited y su uso comercial requiere licencia. La aplicación oficial F1 y la plataforma F1TV ofrecen una interfaz de consumo de estos datos, no una herramienta de análisis.

2.2.2 Ecosistema de código abierto

Frente a las soluciones propietarias, en torno a los datos que la Fórmula 1 publica ha surgido un ecosistema de herramientas de código abierto. Estas no acceden a la telemetría completa de los equipos, sino al subconjunto de datos públicos, pero lo hacen de forma gratuita y documentada. A continuación se describen las principales fuentes de datos sobre las que se sustenta este ecosistema, incluida la que emplea este trabajo.

2.2.2.1 FastF1

FastF1 es la biblioteca de Python de referencia para el acceso y análisis de datos históricos de Fórmula 1 [21]. Desarrollada y mantenida por la comunidad bajo el pseudónimo de theOehrly, alcanza las 5.000 estrellas en GitHub y es la herramienta más utilizada en proyectos de análisis amateur y académico. FastF1 unifica el acceso a dos fuentes: la API no oficial de sincronización en directo de la F1 y Jolpica-F1 para resultados históricos. Toda la información se devuelve como DataFrame de Pandas extendido con métodos específicos del dominio. Las capacidades principales incluyen tiempos de vuelta y sector, telemetría básica del coche correlacionada con posición X/Y en la pista, datos meteorológicos, información de compuestos y un sistema de caché en disco que evita descargas.

La cobertura histórica es fiable desde la temporada 2019, con disponibilidad parcial desde 2018. Las limitaciones más significativas son la dependencia de una API no oficial que F1 podría modificar o restringir sin previo aviso. FastF1 es una librería de acceso y preprocesado, no ofrece ninguna interfaz de usuario.

2.2.2.2 De Ergast a Jolpica-F1

Durante más de una década, la Ergast Motor Racing Developer API [20] fue la fuente canónica de datos históricos de F1 para la comunidad, ofreciendo resultados y clasificaciones desde la temporada

1950 a través de una API REST documentada y gratuita. Ergast fue deprecada a finales de 2024. Como respuesta, la comunidad desarrolló Jolpica-F1 [13], una reimplementación compatible con los mismos *endpoints*, mantenida de forma voluntaria y con cobertura desde 1950 hasta el presente. FastF1 migró a Jolpica como *backend* histórico. La deprecación de Ergast ilustra la fragilidad estructural de toda la cadena de datos abiertos en la que se apoya este trabajo: la disponibilidad de los datos depende de decisiones de terceros sin compromisos de continuidad.

2.2.2.3 OpenF1 API

OpenF1 [22] es una API REST gratuita y de código abierto que proporciona telemetría en tiempo real e histórica. Expone dieciocho *endpoints* con documentación pública completa: telemetría del coche a 3,7 Hz, posición en pista cada cuatro segundos, tiempos y datos del pit lane, mensajes de dirección, registros de radio y datos meteorológicos.

La cobertura histórica se limita a partir de 2023, lo que supone una desventaja frente a FastF1. La versión gratuita cubre todos los *endpoints* históricos sin autenticación, el acceso en directo requiere una suscripción. Esta API representa la evolución más documentada del ecosistema abierto, aunque su cobertura temporal reducida limita su uso para análisis multitemporada.

2.2.3 Proyectos y Herramientas de la comunidad

Sobre las fuentes de datos anteriores, la comunidad ha construido un conjunto de proyectos y herramientas de visualización. Su alcance y madurez son muy variados, desde cuadernos de análisis hasta dashboards interactivos constituyendo el contexto más cercano a este proyecto.

2.2.3.1 Dashboards basados en FastF1

F1Dash (FraserTarbet) [30] es un dashboard analítico interactivo construido con Python y Plotly sobre FastF1. Proporciona competitividad por vuelta, comparación de trazas, telemetría superpuesta entre pilotos y un panel de condiciones de pista filtrable por equipo, piloto y compuesto, como se aprecia en la Figura 2.3. Es el referente más cercano a TALOS dentro del ecosistema abierto aunque no cubre la totalidad de módulos que este trabajo implementa.

F1-Tempo [7] es una herramienta web de comparación de telemetría construida con React y TypeScript sobre FastF1. Permite superponer la telemetría de distintos pilotos, sesiones o años, e incluye un mapa del circuito coloreado según qué piloto fue más rápido en cada punto y la selección automática de las vueltas más rápidas. Es un buen ejemplo de herramienta puesto que resuelve muy bien una única

tarea (la comparación de trazas vuelta a vuelta), pero su alcance se limita a esa función y no aborda el análisis de estrategia, la cronología de eventos, la dinámica vehicular ni la clasificación de estilos que sí cubre este trabajo.

f1-dash (slowlydev) es un dashboard en tiempo real construido con Next.js que consume la API oficial. Proporciona la clasificación en directo, estado de neumáticos y tiempos de sector durante las sesiones. Su orientación es la monitorización en directo, no el análisis post-sesión.

En GitHub, el *topic* fastf1 agrupa decenas de repositorios con cuadernos Jupyter para análisis de temporada. La mayoría son análisis puntuales no mantenidos y sin interfaz de usuario.



Figura 2.3. Interfaz de F1Dash, dashboard de telemetría en tiempo real basado en FastF1.

2.2.3.2 Grafana y el stack de monitorización genérico

Grafana es una plataforma de observabilidad de código abierto orientada a series temporales. Existe un caso en el blog de Grafana Labs [24] en el que se implementa un *stack* de telemetría F1 utilizando el *feed* UDP del videojuego F1 2020 de Codemasters como fuente de datos, Apache Kafka como broker de mensajería, InfluxDB como base de datos y Grafana como visualización, desplegado sobre Kubernetes.

El resultado es impresionante a nivel técnico pero conceptualmente alejado del problema que abarca este trabajo: Los datos provienen de una simulación y la infraestructura necesaria requiere conocimientos significativos. Grafana no incorpora ningún concepto del dominio.

2.2.3.3 Herramientas BI genéricas: Tableau y Power BI

Las principales plataformas de *Business Intelligence* cuentan con adopción documentada en el análisis deportivo de élite para carga de entrenamiento, prevención de lesiones y análisis táctico. En F1, McLaren ha reconocido el uso de Tableau junto a Python y MATLAB como herramientas complementarias.

Sin embargo, ni Tableau ni Power BI incorporan conectores nativos para las APIs anteriormente mencionadas, por lo que el acceso a telemetría requiere pipelines de preprocesado en Python. Las licencias tienen coste. Estas herramientas están optimizadas para métricas de negocio y agregaciones, el análisis de señales de telemetría es un caso de uso atípico que no explotan de forma natural.

2.2.4 Investigación académica

El interés académico y universitario por el análisis de datos ha crecido notablemente desde la disponibilidad de la API FastF1. Los trabajos existentes se agrupan en dos líneas claramente diferenciadas: investigación en predicción y optimización de estrategia, y proyectos de visualización e interfaz de usuario. Ninguno de los trabajos combina ambas líneas de forma integrada.

2.2.4.1 Investigación Internacional en estrategia y predicción

Bekker y Lotz (2009) [4] publicaron uno de los primeros modelos formales de simulación de estrategia de carrera mediante simulación de eventos discretos, validado sobre tres Grandes Premios de 2005. Operan sin datos reales de telemetría puesto que no existían APIs en ese entonces y recurren a modelos paramétricos. Representa el punto de partida de la investigación operativa aplicada a la Fórmula 1. **Heilmeier et al. (2020)** [12] proponen el *Virtual Strategy Engineer*, un sistema basado en redes neuronales artificiales (FFNN + LSTM) entrenado sobre datos de temporadas 2014–2019 para tomar decisiones de pit stop y selección de compuesto. Es uno de los primeros trabajos que aborda el problema de estrategia de carrera desde el aprendizaje profundo y ha servido de referencia para trabajos posteriores. **Aversa, Cabantous y Haeffliger (2018)** [3] analizan en el *Journal of Strategic Information Systems* el Gran Premio de Abu Dabi 2010, donde Ferrari tomó una decisión de estrategia subóptima que costó el campeonato a Alonso a pesar de disponer de herramientas DSS avanzadas. Los autores identifican tres fuentes de fallo: la naturaleza situada de la toma de decisiones, la cognición distribuida y la performatividad del DSS. Este marco teórico justifica el diseño de herramientas transparentes y no prescriptivas como TALOS. **Aguad y Thraves (2024)** [1] formulan la estrategia de pit stops en F1 como un juego de Stackelberg de suma cero con programación dinámica,

modelizando la interacción estratégica entre equipos ante eventos imprevistos como el *safety car*. Publicado en el *European Journal of Operational Research*, representa la aplicación más rigurosa de la teoría de juegos a la estrategia de carreras. En el ámbito del aprendizaje por refuerzo, **Thomas et al. (2025)** [31] presentan en ACM SAC un sistema de RL explicable (*Explainable Reinforcement Learning*) para estrategia de carrera, introduciendo árboles de decisión. Probado sobre el GP de Baréin 2023.

2.2.4.2 Trabajos de fin de grado

El panorama de TFG/TFM en España sobre análisis de Fórmula 1 reproduce a escala universitaria la misma dicotomía que se observa en la investigación.

Morán Montero (Universidad de Extremadura, 2022) [19] desarrolla dos modelos de aprendizaje automático, incluye un clasificador de circuitos y una red neuronal para predecir resultados de carrera. El enfoque es modelado offline sin incluir ninguna interfaz de usuario. No utiliza FastF1 ya que este trabajo es anterior a su adopción generalizada.

Cid Espeso (Universidad Politécnica de Madrid, 2023) [5] construye un dashboard interactivo de seguimiento de carreras utilizando Dash Plotly como framework, FastF1 y OpenF1 para los datos. Usa MongoDB como caché de sesión. Cubre estrategias de *pitstop*, ritmo de carrera y comparativas. Es el referente más cercano a este proyecto entre los TFGs españoles. Comparte el enfoque de interfaz con datos reales. La principal diferencia es la integración de clasificadores y módulos de ML además del análisis de fuerzas G.

Latorre Castelltort (Universitat de Barcelona, 2025) [15] desarrolla una aplicación de análisis histórico y en tiempo real con un chatbot de lenguaje natural para el soporte al usuario.

2.2.4.3 Análisis comparativo y posicionamiento de la aplicación

El recorrido por herramientas profesionales, ecosistema *open-source*, proyectos comunitarios y trabajos académicos permite trazar con precisión el lugar que este proyecto ocupa. La tabla 2.1 resume los atributos de cada alternativa.

Tabla 2.1. Comparativa de herramientas de análisis de telemetría de Fórmula 1.

Herramienta	Gratuita	F1	UI	Cobertura	Barrera
McLaren ATLAS	No	Sí	Sí	Completa (privada)	Restringida
AWS F1 Insights	No	Sí	No	Live (broadcast)	Sin API
MoTeC i2	No	No	Sí	Hardware propio	Alta
FastF1	Sí	Sí	No	2019–hoy	Baja
OpenF1	Parcial	Sí	No	2023–hoy	Baja
F1Dash	Sí	Sí	Sí	FastF1	Baja
Grafana + Kafka	Sí	No	Sí	Simulación	Muy alta
Tableau / Power BI	No	No	Sí	Requiere pipeline	Media
TFG Morán (2022)	Sí	Sí	No	Batch offline	Media
TFG Cid (2023)	Sí	Sí	Sí	2019–hoy	Baja
TALOS	Sí	Sí	Sí	2018–hoy	Baja

3

Metodología, Decisiones de análisis y diseño del sistema

3.1 Metodología

El desarrollo de este proyecto ha seguido un enfoque iterativo e incremental adoptando prácticas habituales en proyectos de software ágil sin ceñirse a un marco estricto de trabajo. Este modelo resulta adecuado para proyectos explorativos en los que los requisitos evolucionan a medida que se profundiza en el dominio del problema.

3.1.1 Modelo de desarrollo

El desarrollo se organizó en iteraciones cortas centradas en la entrega de funcionalidad completa por módulo. Cada iteración seguía un ciclo de cuatro fases: análisis de la funcionalidad, diseño de la solución, implementación y validación del resultado mediante tests unitarios y conocimiento del dominio. Al finalizar cada iteración el módulo quedaba integrado en la aplicación principal y disponible para su uso y posterior revisión. Este enfoque permitió detectar y corregir problemas de forma temprana, permitió pivotar el diseño de módulos posteriores a partir de la experiencia acumulada en los anteriores.

3.1.2 Seguimiento con el tutor

A lo largo del proyecto se mantuvieron reuniones con el tutor. Cada reunión tenía como objetivo revisar el avance desde el último encuentro, validar las decisiones técnicas y establecer los objetivos y mejoras para la siguiente iteración. Este canal de comunicación permitió mantener la coherencia del proyecto y reorientar el trabajo cuando fue necesario.

3.1.3 Gestión de tareas

Para la gestión de tareas se utilizó **GitHub Projects**, configurado como tablero *kanban* con cinco columnas: *backlog*, *ToDo*, *In Progress*, *Done* y *Testing*. Cada tarea correspondía a un módulo, funcionalidad o conjunto de módulos durante el desarrollo. Además, las diferentes tareas fueron divididas en épicas que corresponden al marco general de la funcionalidad: Algoritmos, Diseño, *Backend*, Interfaz gráfica, Pruebas/validación y Entregables. La Figura 3.1 muestra el estado del tablero durante el desarrollo.

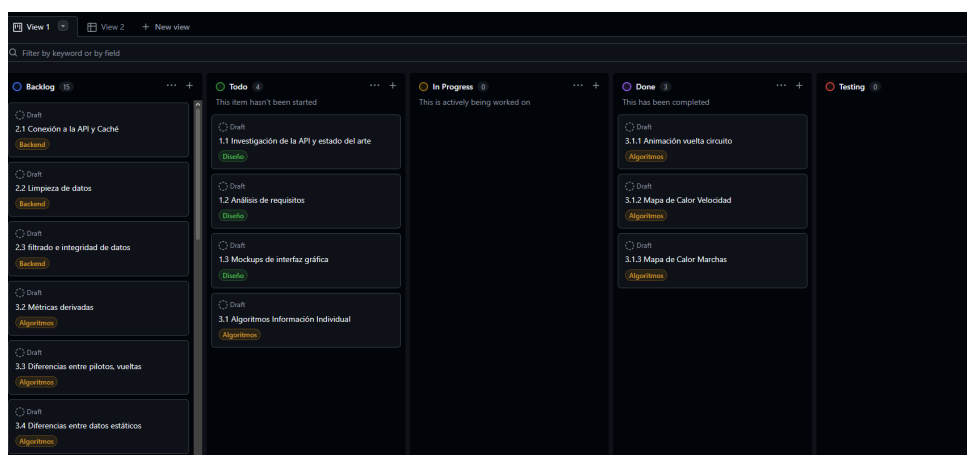


Figura 3.1. Ejemplo de gráfico GitHub Projects durante la realización del trabajo.

3.1.4 Control de versiones

El código fuente se gestionó con *Git* y se alojó en un repositorio de *GitHub*. Se trabajó sobre la rama principal *main*, realizando commits asociados a funcionalidades o conjunto de estas. El repositorio es público y está disponible en <https://github.com/hgdario/f1-telemetry-dss>. El proceso de instalación y puesta en marcha se detalla en el Apéndice A.

3.1.5 Planificación temporal

La tabla 3.1 recoge la distribución de horas por fase establecidas en el anteproyecto.

Tabla 3.1. Temporización del proyecto por fases.

Fase	Horas
Planificación y estado del arte	30
Diseño de la arquitectura y datos	55
Desarrollo de algoritmos de análisis y métricas	40
Programación de la lógica para el cálculo de métricas	35
Implementación del dashboard y controladores	30
Integración de visualizaciones dinámicas	35
Elaboración de pruebas y validación	35
Elaboración de memoria técnica	36
Total	296

3.2 Análisis de requisitos

Como se indicó en el anteproyecto de este proyecto se ha realizado un trabajo profundo de análisis de requisitos. Esta investigación sobre el diseño de la aplicación nos deja con los siguientes requisitos funcionales y no funcionales: A cada requisito se le ha asignado una prioridad basada en la importancia dentro del dominio del problema y la dificultad/interés técnico que profesa. La tabla 3.2 representa los requisitos funcionales del sistema mientras que la 3.3 los no funcionales.

Tabla 3.2. Requisitos funcionales del sistema.

ID	Nombre	Descripción	Prioridad
RF01	Selección de temporada	El sistema permite seleccionar año	Alta

Continúa en la página siguiente

RF02	Selección de sesión	El sistema permite seleccionar GP y sesión eligiendo entre <i>Free Practice</i> , <i>Qualy</i> o carrera. Si lo hubiera se incluyen los <i>sprints</i>	Alta
RF03	Carga de telemetría	Descargar y subir a caché los datos completos de la sesión que se descarga	Alta
RF04	Resumen de sesión	El primer módulo del sistema, mostrar circuito, clasificación y datos generales	Baja
RF05	Cronología de Eventos	Línea temporal de actividad durante la sesión, balance de adelantamientos y tiempo de paradas en <i>boxes</i>	Media
RF06	Mapa de Velocidad	Mapa de calor del circuito según la velocidad	Baja
RF07	Mapa de Marchas	Mapa de calor del circuito según la marcha del piloto	Baja
RF08	Estrategia de neumáticos	Diagrama de Gantt indicando los compuestos usados. Se debe indicar la combinación más usada	Media
RF09	Telemetry Trace	Canales de telemetría completos de una vuelta a elegir	Alta
RF10	Resumen del piloto	KPIs, sectores y stints de cada piloto	Baja
RF11	Panel de equipo	Panel con sendos pilotos e información para un equipo	Alta
RF12	Comparativa de sesión	Canales de telemetría multi-piloto	Media
RF13	Animación circuito	Animación del coche por el circuito sincronizada a tiempo	Alta
RF14	Vuelta Ideal	Vuelta óptima dividiendo el circuito en microsectores, permite filtrado por equipo	Media

Continúa en la página siguiente

RF15	Coche Fantasma	Comparación de dos monoplazas en una animación conjunta	Alta
RF16	Diagrama G-G	Análisis de fuerzas G del trazado y Diagrama G-G teórico de la vuelta	Alta
RF17	Carga aerodinámica	Estimación teórica del downforce	Baja
RF18	Carga por rueda	Mapa de calor del circuito según la carga del neumático	Alta
RF19	Clasificador de tipo de vuelta	Uso de <i>K-means</i> para agrupar en <i>clusters</i> los tipos de vuelta	Alta
RF20	Clasificador de estilo de conducción	Uso de <i>K-means</i> para agrupar en <i>clusters</i> los tipos de conducción basado en tipo de vuelta y telemetría de vuelta	Alta
RF21	Clasificador de tipo de circuito	Uso de <i>K-means</i> para agrupar en <i>clusters</i> los tipos de circuito basado en fuerzas G	Alta

Tabla 3.3. Requisitos no funcionales del sistema.

ID	Nombre	Descripción	Prior.
RNF01	Rendimiento	Carga reducida mediante caché de disco y en memoria	Alta
RNF02	Usabilidad	Interfaz navegable sin conocimientos técnicos	Alta
RNF03	Reproducibilidad	Instalable con un único bat desde el repositorio	Alta
RNF04	Ejecutabilidad	Ejecutable con un único bat desde el repositorio	Alta
RNF05	Independencia	Sin APIs de pago, servidores propios ni BBDD externas	Alta
RNF06	Compatibilidad	Ejecutable en cualquier SO con Python 3.10+	Media
RNF07	Mantenibilidad	Módulos independientes fácilmente ampliables	Media
RNF08	Código abierto	Código fuente público y libre para su uso.	Media

3.3 Tecnologías empleadas

El desarrollo se ha llevado íntegramente en **Python**, aprovechando el ecosistema de bibliotecas científicas y de visualización que lo convierten en el entorno natural para proyectos de este estilo. La interfaz de usuario ha sido construida sobre **Streamlit** [29], un *framework* de código abierto que permite desarrollar aplicaciones web interactivas orientadas al análisis de datos y sin separar el *backend* del *frontend*. Los gráficos e interacciones visuales han sido implementados con **Plotly** [26], que proporciona visualizaciones integrables de forma nativa en Streamlit.

El acceso a los datos de la Fórmula 1 se realiza a través de **FastF1**, una biblioteca de código abierto que unifica el acceso a la API oficial de la Fórmula 1 y a la API histórica de Ergast, incorporando además un sistema de caché local que reduce los tiempos de carga en sesiones repetidas. El procesamiento y la manipulación de los datos se apoyan en **Pandas** [16] y **NumPy** [11], mientras que el filtrado de señales de telemetría utiliza la implementación del filtro Savitzky-Golay disponible en **SciPy** [32]. Los módulos de aprendizaje automático se implementan con **scikit-learn** [25] para el entrenamiento y **NumPy**. El control de versiones y el repositorio del proyecto se alojan en **GitHub**. El entorno de desarrollo integrado utilizado ha sido **Visual Studio Code** y la elaboración de diagramas y esquemas de la memoria se ha realizado con **draw.io**. La Figura 3.2 recoge el conjunto de tecnologías empleadas.



Figura 3.2. Tecnologías utilizadas en el desarrollo de TALOS.

3.4 Diseño del sistema

A la hora de planear el diseño del sistema se han valorado alternativas en casi todos los ámbitos del desarrollo desde el *framework* de interfaz hasta las decisiones de gráficos y clasificadores. En esta sección recorreremos de forma granular las decisiones de diseño más importantes que se han tomado durante la planificación y desarrollo de la aplicación. TALOS se organiza en módulos de análisis agrupados en cuatro categorías que siguen una progresión de lo general a lo específico, tal y como se muestra en la Tabla 3.4.

Tabla 3.4. Organización de módulos de TALOS por categoría.

Categoría	Módulos
Información de Sesión	Resumen de sesión, Cronología de eventos, Estrategia de neumáticos
Análisis de Piloto	Resumen del piloto, Mapa de velocidad, Mapa de marchas, Telemetry trace
Comparativas	Panel de equipo, Comparativa de sesión, La vuelta ideal, Coche fantasma
Dinámica Vehicular	Diagrama G-G, Carga aerodinámica, Mapa de carga por rueda

La Figura 3.3 muestra el flujo de navegación entre vistas y la relación de componentes de manera ilustrada para facilitar la comprensión.

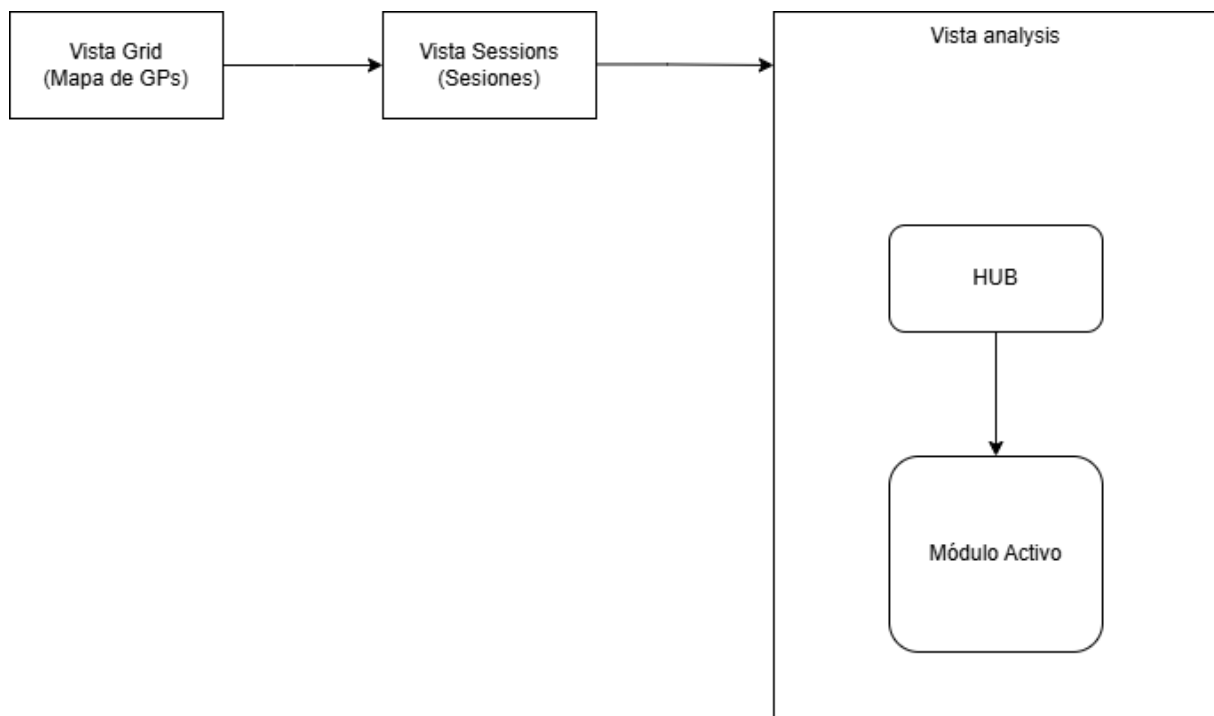
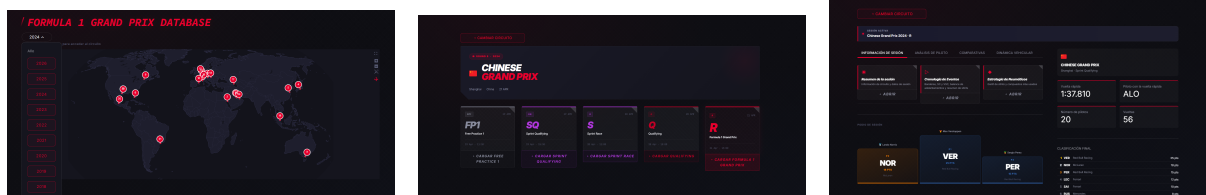


Figura 3.3. Diagrama de navegación y arquitectura de vistas de TALOS.

Sigue una arquitectura de *Single Page Application* construida sobre Streamlit. La navegación se articula en tres vistas secuenciales: la vista en `grid` (Figura 3.4a), que representa el mapa mundial interactivo que aloja los Grandes Premios de la temporada seleccionada. La vista `sessions` (Figura 3.4b), donde el usuario elige el tipo de sesión a analizar y por último el `analysis` (Figura 3.4c) que aloja el hub de módulos y el módulo activo en cada momento. La transición entre estas vistas se gestiona mediante la función `go()` de `session_loader.py`, que actualiza el estado de sesión y fuerza la re-ejecución del script.



(a) Vista Grid

(b) Vista Sessions

(c) Vista Analysis

Figura 3.4. Las tres vistas secuenciales de la navegación de TALOS: el mapa mundial de Grandes Premios (Grid), la selección de sesión (Sessions) y el hub de módulos de análisis (Analysis).

3.4.1 FastF1 sobre otras alternativas

Como hemos comentado antes la API oficial de la Fórmula 1 requiere suscripción de pago volviéndola incompatible con el enfoque *open-source*. Otras alternativas como OpenF1 solo tiene cobertura desde 2023 por lo que cuenta con muchos canales de telemetría incompletos. FastF1 es la solución gratuita con más cobertura, es una API que está documentada y unifica la API oficial y datos históricos con la integración de Ergast y Jolpica, contando con una comunidad muy activa en GitHub. El sistema de caché de disco es un factor diferencial con el objetivo de reducir descargas repetidas sin aportar nada adicional.

3.4.2 Streamlit para interfaz

Para la interfaz se ha usado Streamlit, un *framework* de ciencia de datos que permite construir la aplicación íntegramente en Python sin depender de una separación entre *backend* y *frontend*. El modelo de re-ejecución completa en cada interacción encaja bien con los dashboards de análisis donde el usuario cambia parámetros y espera nuevos resultados. Tiene una curva baja de aprendizaje lo que permitió centrarse en la lógica de análisis en lugar de la infraestructura web. Streamlit permite la ejecución en local mediante un único comando sin configurar servidores, puertos ni builds. Sin embargo, Streamlit cuenta con un menor control sobre la interfaz gráfica del proyecto permitiendo una limitada personalización visual. Esto pudo compensarse con inyecciones de CSS desde un archivo aparte de assets. Se valoraron otras alternativas como Dash (más flexible) y Flask+JS sin embargo es mucho más complejo.

3.4.3 Estado de la sesión

Streamlit no tiene estado persistente entre re-ejecuciones salvo `st.session_state`. Se centraliza en esta sesión todo lo que las vistas requieren compartir como año seleccionado, evento, objeto a tratar o módulo activo. Esto permite que independientemente de dónde se encuentre el usuario en la aplicación se compartan los datos más importantes entre módulos. `session_loader.py` es el único punto que escribe el estado inicial evitando inconsistencias. Si un módulo necesita información de la sesión lee el estado en lugar de pedirla de nuevo.

3.4.4 Caché de dos niveles

Se usa un primer nivel de caché en disco. FastF1 serializa los datos de cada sesión que ha sido cargada en disco, cuando se trate de cargar la misma sesión no hace ninguna petición de red lo que permite a la aplicación funcionar offline. En un segundo nivel se utiliza `@st.cache_data` en `session_loader.py` lo que permite mantener el calendario y la sesión en RAM durante la ejecución así no necesita traer de disco cada interacción. Sin este sistema de doble caché, cada cambio de módulo relanzaría la descarga completa de telemetría.

3.4.5 Módulos independientes entre sí

Se ha tratado de reducir lo máximo posible el acoplamiento entre módulos permitiendo así añadir, modificar o eliminar un módulo sin riesgo de romper los demás. La mayoría de los módulos importan solo de `session_loader` y librerías externas. Hay un router principal `app.py` que lanza la aplicación y actúa como orquestador. Este *router* lee `active_module` del estado y llama a la función `render_*`() correspondiente. Esto permite que los módulos puedan desarrollarse y probarse independiente y en paralelo.

3.4.6 K-Means pre-entrenado

A la hora de realizar los clasificadores se ha optado por tener pre-entrenados estos. Entrenar los clasificadores en tiempo real requeriría descargar y procesar varias temporadas de datos resultando inviable en tiempo de respuesta. Los scripts de entrenamiento están separados de la parte de producción de modo que ni se importan ni ejecutan. El modelo entrenado es cargado en runtime de forma rápida,

es esta separación la que permite re-entrenar los clasificadores con datos nuevos sin necesidad de tocar el código de la aplicación.

3.4.7 Interfaz Visual

El diseño visual adopta una estética oscura inspirada en la identidad gráfica de la Fórmula 1. Consta de una paleta basada en tonos de rojo #E8002D sobre fondos oscuros. Uso de la tipografía Titillium Web habitual en la comunicación del deporte (además de Inter y JetBrains Mono). El objetivo era que la herramienta resultara familiar desde el primer vistazo a cualquier aficionado promedio, sin instrucciones necesarias.

Antes de comenzar la implementación se elaboraron *mockups* de las vistas principales usando draw.io, estos bocetos recogían la disposición general de los elementos aunque algunos acabaron pivotando en el layout. Han servido como referencia visual temprana durante el desarrollo. El funcionamiento completo de la interfaz y la descripción de cada uno de sus módulos se recogen en el Apéndice B.

4

Implementación

Los capítulos anteriores han descrito qué es lo que hace la aplicación y por qué se tomaron determinadas decisiones de diseño. Este capítulo da el paso siguiente explicando cómo se han materializado todas las decisiones de diseño en código. Se mostrarán fragmentos más representativos de determinadas decisiones técnicas en lugar de mostrar el código completo, que se encuentra en el repositorio.

Como se detalló en el apartado de tecnologías, el sistema se ha construido en Python, empleando Streamlit y FastF1 como fuente de datos, Plotly para la visualización y CSS y HTML para la interfaz de usuario. A lo largo del capítulo se recorren los distintos subsistemas de la herramienta: Primero la estructura general del código y la organización de los módulos, la gestión de estado y caché, procesamiento de telemetría y cálculo de variables físicas y los clasificadores no supervisados. Por último se detallará el sistema visual que da su identidad a la aplicación.

4.1 Estructura del código

El código de la aplicación se organiza en dos raíces claramente diferenciadas siendo `src/` y `research/`. La primera, contiene el código en producción: todo lo que se ejecuta cuando un usuario lanza la aplicación. La segunda, agrupa los *scripts* de exploración, contando con versiones antiguas de módulos ya implementados o nuevos módulos en progreso de implementar. Esta carpeta tiene como intención el tener un entorno de investigación para un futuro desarrollo o mejora de lo ya programado. Esta carpeta también contiene los *scripts* de entrenamiento que se ejecutan una sola vez y de forma aislada, sin formar parte de la herramienta desplegada. Esta separación es deliberada de modo que se mantienen los prototipos fuera del código de producción evitando que la lógica experimental contamine la aplicación y deja claro qué es lo que se ejecuta en cada contexto.

El listado 4.1 muestra la estructura de directorios del proyecto, tal y como se encuentra en el repositorio.

```
TALOS/
|-- src/                # Código de producción
|   |-- app.py          # Punto de entrada y enrutado
|   |-- session_loader.py # Estado, carga y cache
|   |-- ui_assets.py    # CSS, JS y utilidades de color
|   |-- Circuit2d.py    # Renderizado 2D del trazado
|   |-- CircuitClassifier.py # Clasificador de circuitos (K-Means)
|   |-- DriverStyleClassifier.py # Clasificador de estilo de piloto (K-Means)

|   |-- LapTypeClassifier.py # Clasificador de tipo de vuelta (K-Means)
|   |-- informacion_sesion/ # Resumen, cronología, estrategia
|   |-- analisis_piloto/    # Mapas, telemetría, resumen de piloto
|   |-- comparativas/       # Head-to-head, vuelta ideal, coche fantasma
|   |-- dinamica_vehicular/ # Diagrama G-G, aero, carga por rueda
|   |-- assets/             # Logos de equipos y siluetas
|   \-- data/               # Firmas de pilotos precalculadas
|-- research/              # Entrenamiento offline (scrapers + K-Means)
|   \-- data/              # Datasets de características
|-- requirements.txt       # Dependencias con versiones fijadas
|-- install.bat / start.bat # Instalación y arranque (Windows)
\-- f1_cache/              # Cache de FastF1 (excluida de git)
```

Listado 4.1. Estructura de directorios del proyecto.

El punto de entrada a la aplicación es `app.py`, que actúa como orquestador y no contiene lógica de análisis. Su responsabilidad se limita a coordinar definiendo la navegación, importando los estilos globales, inicializa el estado de sesión y delega cada vista al módulo correspondiente. El catálogo de funcionalidades se declara en un solo diccionario `NAV`, que constituye la fuente de verdad sobre qué módulos existen, a qué categoría pertenecen y con qué icono se presentan. Gracias a este registro incorporar una nueva funcionalidad se reduce a añadir una entrada en el `NAV` y su correspondiente `import`, sin tocar la lógica del enrutado manteniendo al mínimo el acoplamiento entre componentes. El listado 4.2 muestra la separación en categorías dentro del `NAV`.

```
1 NAV = {
2     "Información de Sesión": [
```

```

3      ("Resumen de la sesión", "...", "Información de circuito y datos de
      sesión"),
4      ("Cronología de Eventos", "...", "Banderas, SC y VSC, balance de
      adelantamientos y resumen de stints"),
5      ("Estrategia de Neumáticos", "...", "Gantt de stints y compuestos más
      usados"),
6  ],
7  "Análisis de Piloto": [
8      ("Resumen del Piloto", "...", "Síntesis de sesión: KPIs, sectores,
      stints y distribución"),
9      ("Mapa de Velocidad", "...", "Circuito coloreado por velocidad (km
      /h)"),
10     ("Mapa de Marchas", "...", "Circuito coloreado por marcha
      seleccionada"),
11     ("Telemetry Trace", "...", "Canales de telemetría, circuito
      animado y estilo de conducción del piloto"),
12 ],
13 "Comparativas": [
14     ("Panel de Equipo", "...", "Análisis intra-equipo"),
15     ("Comparativa de Sesión", "...", "Overlays multi-piloto sobre la
      misma vuelta"),
16     ("La Vuelta Ideal", "...", "Microsectores óptimos de toda la
      parrilla y filtrado por equipos"),
17     ("Coche Fantasma", "...", "Animación 2D de persecución entre
      pilotos"),
18 ],
19 "Dinámica Vehicular": [
20     ("Diagrama G-G", "...", "Círculo de tracción y envolvente de
      Gs"),
21     ("Carga Aerodinámica", "...", "Estimación teórica de carga
      aerodinámica"),
22     ("Mapa de Carga por Rueda", "...", "Distribución de carga por eje y
      temperatura de frenos"),
23 ],

```

```

24 }
25
26 # Lista aplanada de módulos, usada por el enrutado
27 MODULE_KEYS = [m for cat in NAV.values() for m, _, _ in cat]

```

Listado 4.2. Registro declarativo de la navegación en *app.py*.

Los módulos se reparten en cuatro paquetes correspondientes a las cuatro categorías presentadas: *informacion_sesion/*, *analisis_piloto/*, *comparativas/* y *dinamica_vehicular/*. Esta correspondencia directa facilita localizar cualquier funcionalidad basándonos en la categoría que el usuario ve. Todos los módulos comparten además una función pública llamada `render_*`() que recibe la sesión ya cargada y se encarga de construir su vista. Esta uniformidad hace que, desde el orquestador, todos los módulos sean intercambiables ya que responden a la misma firma.

En la raíz de *src/* quedan únicamente los componentes compartidos por toda la aplicación siendo *session_loader.py* que centraliza la gestión del estado, la carga de sesiones y la caché, mientras que *ui_assets.py* concentra los estilos y las utilidades de color. Junto a ellos residen los clasificadores, que se implementan como utilidades en lugar de como vistas ya que no aparecen en el **NAV** anteriormente mencionado y son los propios módulos los que invocan a los clasificadores.

El repositorio se ha preparado pensando en la reproducibilidad. El archivo *requirements.txt* fija las versiones exactas de las dependencias, y los *scripts* de instalación automatizan la instalación y el arranque en sistemas Windows. La caché de telemetría de FastF1 se almacena en *f1_cache/* un directorio excluido del control de versiones ya que no tiene lugar versionar datos descargables que se regeneran la primera vez que se solicita la sesión.

4.2 Modelo y Acceso a datos

Obtenemos todos los datos de una única fuente: **FastF1**, una biblioteca Python que combina la API de Fórmula 1, la telemetría oficial y datos de Ergast. No hay base de datos propia ni scraping manual, FastF1 abstrae la descarga y parseo y devuelve *DataFrames* de Pandas listos para operar.

4.2.1 Origen y naturaleza de datos

FastF1 es la única fuente, expone los datos de telemetría a 3,7 Hz aproximadamente (muestreo variable según el canal) y los eventos de *timing* (banderas, stints, tiempos de vuelta, posiciones) en un *DataFrame* de Pandas. Se accede a todo a través de una sesión. Una sesión es una carrera, clasificación, libre o *sprint* de un gran premio y año concretos. La carga de estos eventos se dispara con `fastf1.get_session(year, gp, ses)` seguido de un `.load()`. Este último es la operación red + parseo a JSON que la caché de la aplicación hace opaca para el usuario.

4.2.2 Modelo Jerárquico de datos

FastF1 estructura una sesión en tres niveles anidados que son recorridos en el mismo orden: `Session` expone `session.results` que nos otorga la clasificación final y `session.weather_data` que nos da la temperatura, lluvia y datos meteorológicos.

Los canales de telemetría vienen con *timestamps* relativos al inicio de vuelta, no con distancia recorrida. El listado 4.3 es un ejemplo de cómo vienen los datos dentro de FastF1.

```
# session.laps -- DataFrame, una fila por vuelta
{ "Driver": "VER", "LapNumber": 42, "LapTime": "0:01:31.456",
  "Compound": "MEDIUM", "TyreLife": 8, "Stint": 2 }

# lap.get_telemetry().add_distance() -- ~3.7 Hz, una fila por muestra
{ "Speed": 312.4, "RPM": 11823, "nGear": 7,
  "Throttle": 100.0, "Brake": false, "DRS": 12,
  "X": -2134.5, "Y": 891.2, "Distance": 1456.3 }

# session.results -- una fila por piloto
{ "Abbreviation": "VER", "GridPosition": 1,
  "ClassifiedPosition": "1", "Status": "Finished" }

# session.weather_data -- una fila por muestra meteorologico
{ "AirTemp": 27.4, "TrackTemp": 41.1,
  "Humidity": 38.0, "WindSpeed": 2.1 }
```

Listado 4.3. Campos representativos de cada nivel de datos de FastF1.

4.2.3 Patrón de preparación

Cada módulo sigue la misma secuencia para ir del objeto sesión a los datos a visualizar:

1. **Selección de vuelta** - `laps.pick_drivers(driver)` filtra las vueltas por piloto.
2. **Expansión de telemetría** - `lap.get_telemetry()` devuelve el *DataFrame* de alta frecuencia para esa vuelta
3. **Añadir distancia** - `.add_distance()` integra la velocidad para generar una columna de Distancia que se convierte en el eje X natural de todos los trazados de telemetría.
4. **Saneamiento por canal** - `series.dropna()` descarta muestras ausentes a raíz de fallos o eventos de la carrera como accidentes o banderas rojas.

El listado 4.4 muestra este patrón tal y como aparece en `Telemetrytrace.py`, la función encargada de descargar la telemetría de una vuelta concreta:

```
1 def _extract_telemetry(session: Session, driver: str,
2                             lap_number: int) -> Optional[pd.DataFrame]:
3     """
4     Descarga la telemetría de una vuelta específica y añade columna
5     Distance.
6     Retorna DataFrame o None si falla.
7     """
8     try:
9         laps = session.laps.pick_drivers(driver)
10        lap: Lap = laps[laps["LapNumber"] == lap_number].iloc[0]
11        tel = lap.get_telemetry().add_distance()
12        if tel.empty:
13            return None
14        return tel
15    except Exception as e:
16        st.warning(f"[!] No se pudo cargar la telemetría: {e}")
17        return None
```

Listado 4.4. Patrón de acceso a telemetría en `Telemetrytrace.py`.

4.2.4 Caché

Cargar la sesión tarda entre 2 y 15 segundos según la sesión y la conexión. Sin caché, cada *rerun* de Streamlit relanzaría la descarga. Con `@st.cache_data`, la primera carga persiste en memoria del servidor y por tanto cada *rerun* posterior es instantáneo. El módulo `session_loader.py` centraliza tanto la lógica de caché como el estado de la aplicación. `init_session_state()` registra con `setdefault` todas las claves que la app necesita, de modo que los reruns no pisan el estado ya cargado. `load_schedule(year)` utiliza una caché con `ttl=3600` porque el calendario puede actualizarse a lo largo de la temporada. `load_f1_session(year, gp, ses)` utiliza una caché sin TTL porque una sesión histórica es inmutable; la clave de caché es la tupla `(year, gp, ses)`. Por último, `require_session()` actúa de *guard* en cada módulo: si no hay sesión cargada muestra el aviso y aborta el render como puede observarse en el listado 4.5.

```
1  import streamlit as st
2  import pandas as pd
3  import fastf1
4
5
6  # --- ESTADO
   -----
7
8  def init_session_state() ->None:
9      """Inicializa todas las claves de session_state que usa la app."""
10     st.session_state.setdefault("view",      "grid") # grid / sessions /
        analysis
11     st.session_state.setdefault("selected_year", 2024)
12     st.session_state.setdefault("selected_event", None)
13     st.session_state.setdefault("f1_session", None)
14     st.session_state.setdefault("session_loaded", False)
15     st.session_state.setdefault("session_label", "")
16     st.session_state.setdefault("active_module", None) # None = Hub
17     st.session_state.setdefault("corners", False)
18
19
20 # --- CACHE
```

```

-----
21
22 @st.cache_data(show_spinner=False, ttl=3600)
23 def load_schedule(year: int) ->pd.DataFrame:
24     """Calendario FastF1 del año (sin testing)."""
25     return fastf1.get_event_schedule(year, include_testing=False)
26
27
28 @st.cache_data(show_spinner=False)
29 def load_f1_session(year: int, gp: str, ses: str):
30     """Carga (y cachea) una sesión FastF1. Cachea por (year, gp, ses)."""
31     s = fastf1.get_session(year, gp, ses)
32     s.load()
33     return s
34
35
36 # --- HELPERS
-----
37
38 def require_session() ->bool:
39     """Devuelve False y muestra aviso si no hay sesión cargada."""
40     if not st.session_state.get("session_loaded", False):
41         st.warning("Carga una sesión desde el panel lateral para continuar."
42                 )
43         return False
44     return True
45
46 def go(view: str) ->None:
47     """Cambia la vista activa y fuerza rerun."""
48     st.session_state["view"] = view
49     st.rerun()

```

Listado 4.5. Gestión de estado y caché en `session_loader.py`.

4.3 Información de sesión

La primera de las cuatro categorías funcionales ofrece la visión panorámica de un Gran Premio. A diferencia de las otras categorías, aquí el trabajo consiste en leer los datos que FastF1 entrega y presentarlos de forma que se entiendan con un simple vistazo. La categoría agrupa tres módulos que sirven como resumen de lo que ha ocurrido en un Gran Premio: el resumen de la sesión, cronología de eventos y estrategia de neumáticos.

4.3.1 Resumen de la sesión

El módulo `SessionSummary.py` construye la portada de la carrera, reuniendo en una sola pantalla la información contextual requerida para comprender el Gran Premio. Incluye una cabecera con el nombre del evento y el año, un mapa del trazado del circuito, las condiciones meteorológicas (En el caso de haber cargado una carrera serán relativas a otros fines de semana) y un bloque de estadísticas de la sesión.

El mapa del circuito no es una imagen cargada, es una gráfica mostrada en tiempo real a partir de las coordenadas de posición X e Y contenidas en la telemetría de la vuelta más rápida de la sesión, siguiendo el patrón de acceso a datos descrito en la sección anterior. La propia traza GPS del coche define la silueta del circuito. El bloque meteorológico promedia los registros de `session.weather_data` para mostrar la temperatura del aire, asfalto y humedad siendo estas tres variables que condicionan directamente el rendimiento de los neumáticos y, por tanto, la estrategia. Para añadir un mayor marco contextual, si el evento elegido es de tipo carrera se mostrará una comparación con el resto de fines de semana para poder situar el evento entre los más cálidos o fríos del año. Esta distinción se hace mediante la lectura de `track_temp_by_weekend.csv` localizado en la carpeta de `data/`.

Finalmente, este resumen incorpora lo que en la herramienta se denomina *ADN del circuito*: una clasificación automática que sitúa el trazado dentro de una familia de circuitos con características físicas similares. Esta clasificación se apoya en un modelo de aprendizaje no supervisado cuya construcción y validación se detallan en la sección 4.7, por lo que aquí únicamente se señala su punto de integración. La Figura 4.1 muestra el aspecto de esta vista.

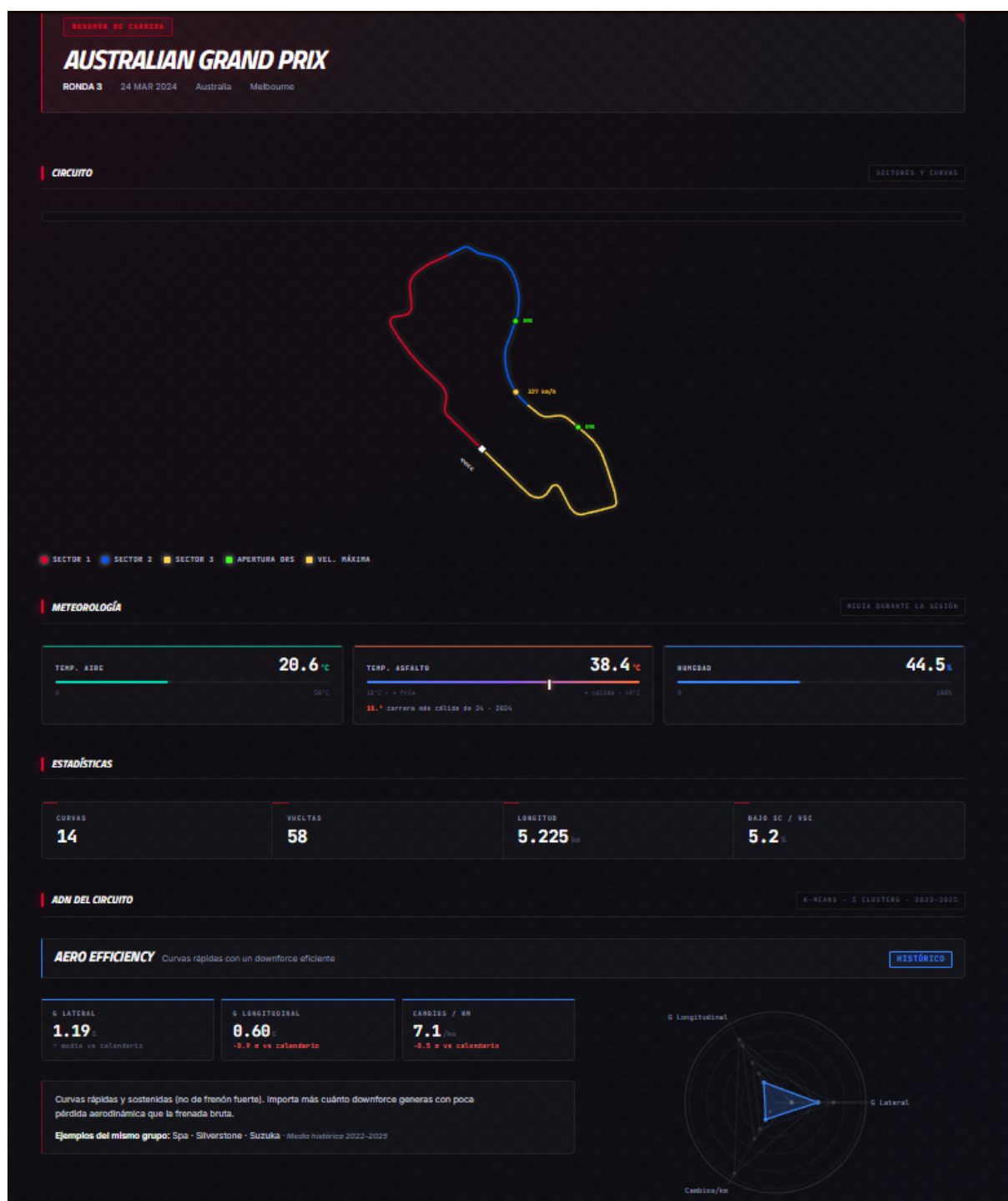


Figura 4.1. Vista de Resumen de la sesión: mapa del circuito dibujado desde la telemetría, condiciones meteorológicas y estadísticas de la carrera.

4.3.2 Cronología de Eventos

Una carrera de Fórmula 1 rara vez transcurre sin interrupciones. Cuando se produce un accidente o aparece un peligro en pista, la dirección de carrera neutraliza total o parcialmente la competición medi-

ante banderas y coches de seguridad. El módulo `SessionTimeLine.py` reconstruye esa secuencia de incidentes a lo largo del tiempo y la presenta como una línea temporal legible.

El reto técnico está en el origen de los datos. FastF1 entrega estos eventos como una secuencia de códigos numéricos en el campo `session.track_status`, un 1 significa pista despejada, un 4 indica un *Safety Car* (Un coche real que encabeza al grupo a una velocidad reducida para parar la carrera), un 6 corresponde al *Virtual Safety Car* (Un equivalente electrónico, sin coche físico), un 5 indica una bandera roja (Sesión suspendida) y así sucesivamente. El módulo traduce cada uno de estos códigos a un evento comprensible asociando a cada código una etiqueta, color y descripción. De este modo un dato pensado para una máquina se convierte en información que cualquier aficionado interpreta sin esfuerzo.

A esta línea temporal se añaden dos análisis complementarios, balance de adelantamientos, que es obtenido comparando la posición de salida con la final, ambas disponibles en `session.results`. El segundo es un resumen de los *stints* de cada piloto. Se entiende por *stint* cada tanda de vueltas consecutivas realizadas con un mismo juego de neumáticos. Es decir, el tramo de carrera entre dos paradas en boxes. La Figura 4.2 muestra esta vista de cronología.

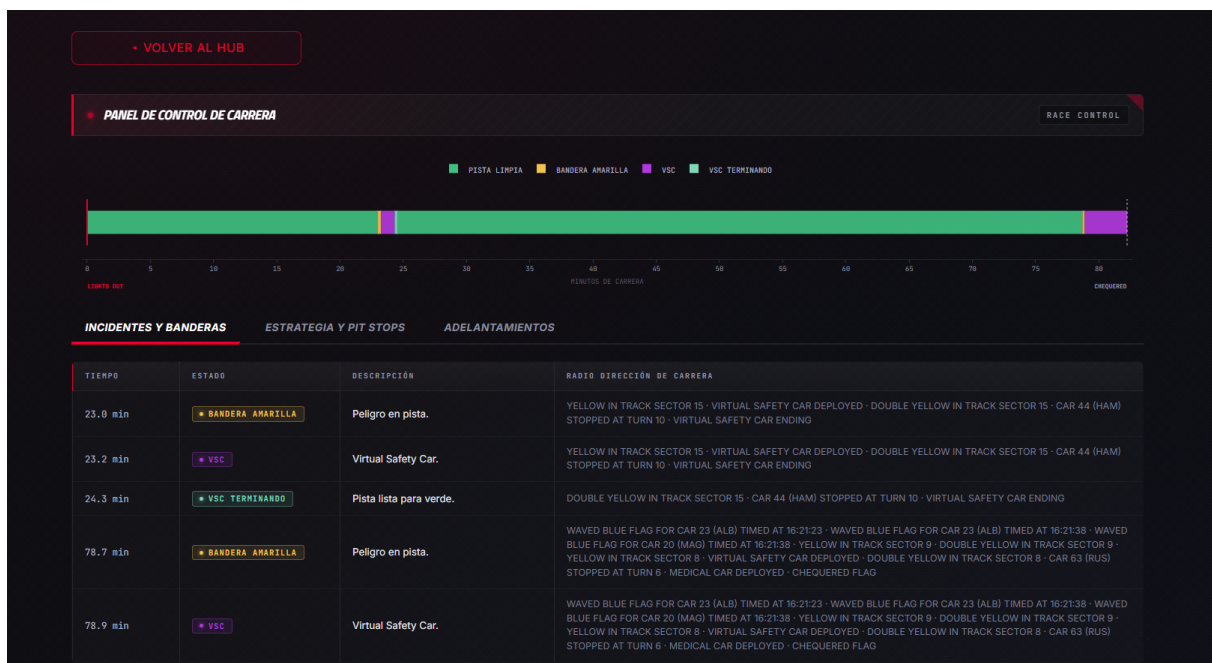


Figura 4.2. Vista de Cronología de Eventos: reconstrucción de la secuencia de banderas, Safety Car y VSC a lo largo de la sesión a partir de los códigos de `track_status`.

A modo de ejemplo, la Figura 4.3 recoge el accidente que originó uno de estos eventos de neutralización en la cronología de la Figura 4.2.

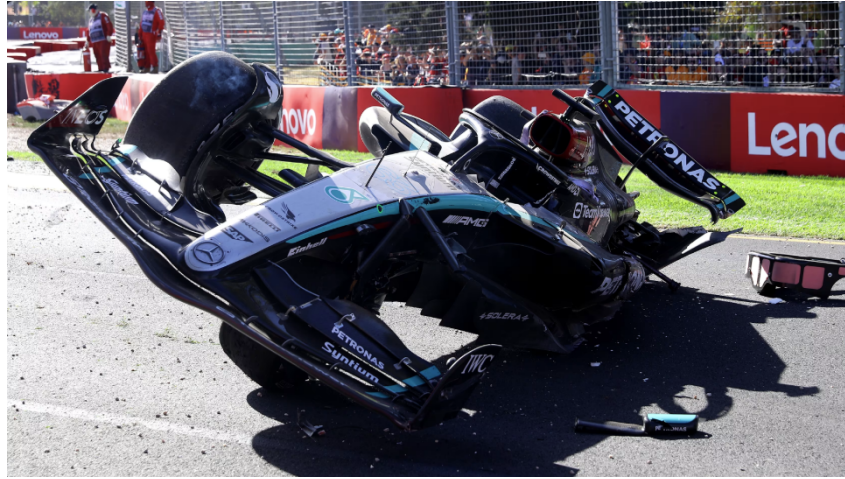


Figura 4.3. Accidente de George Russell en la última vuelta del Gran Premio de Australia 2024, que neutralizó la carrera con bandera amarilla y Virtual Safety Car. Fuente: Formula 1 [8]. Imagen reproducida con fines ilustrativos de carácter académico.

4.3.3 Estrategia de neumáticos

La elección y gestión de los neumáticos es uno de los factores estratégicos más decisivos de la Fórmula 1. Cada equipo dispone de varios *compuestos* (Tipos de goma que van desde el más blando, más rápido pero de mayor desgaste, al más duro, más lento pero más duradero). Decidir cuándo montar cada uno define en buena medida el resultado de la carrera. El módulo `Strat_Grid.py` resume la estrategia de neumáticos de toda la parrilla a través de cuatro visualizaciones.

La primera identifica, para cada compuesto, la vuelta más rápida lograda con él y el piloto que la firmó. La segunda es un gráfico de anillo con el reparto total de vueltas entre los distintos compuestos, que muestra de un vistazo qué neumáticos predominaron en la sesión. La tercera es un diagrama de Gantt que representa, mediante barras por piloto, la secuencia de *stints* y neumáticos utilizados a lo largo de la carrera, de modo que la estrategia de cada equipo queda expuesta visualmente. La cuarta analiza la degradación enfrentando el tiempo por vuelta `LapTime` a la vida del neumático `TyreLife`, evidenciando cómo el rendimiento de la goma decae a medida que acumula kilómetros. Sobre los tiempos se traza para cada compuesto una recta de tendencia mediante regresión lineal cuya pendiente estima el ritmo de degradación en segundos por vuelta, el punto donde se cruzan las rectas de los dos compuestos más utilizados señala el momento en el que el compuesto decae y uno que, a priori es más lento, pasa a ser más rápido. Este fenómeno es conocido como *crossover*. Otro factor importante es el consumo de combustible. Un monoplaza arranca la carrera con hasta 110 kg

de combustible que va quemando progresivamente, de modo que el coche se aligera vuelta a vuelta y, por tanto, rueda más rápido. Cada kilogramo a bordo penaliza el tiempo por vuelta en torno a 0,03 s/kg [6]. Estas cuatro visualizaciones se recogen en la Figura 4.4.

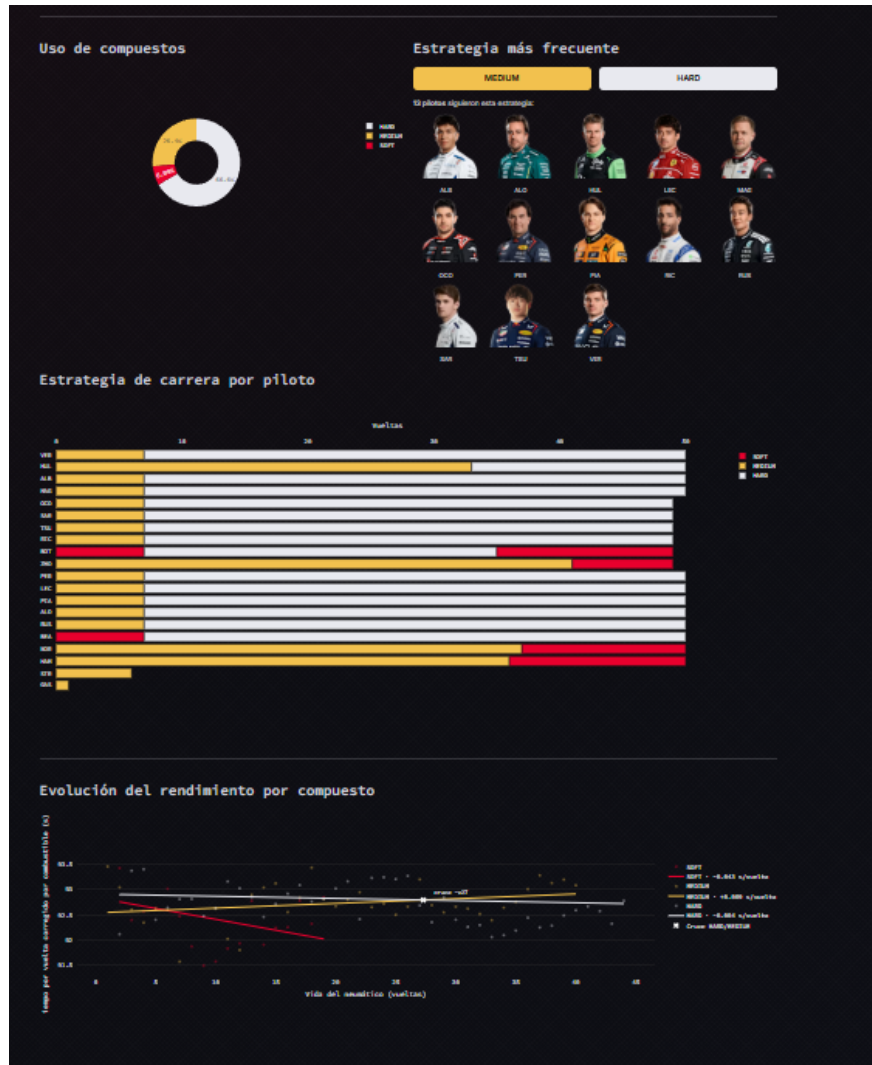


Figura 4.4. Vista de Estrategia de Neumáticos: diagrama de Gantt de stints por piloto, reparto de compuestos y curva de degradación.

4.4 Análisis de piloto

Si la categoría anterior ofrecía la vista de conjunto de un fin de semana, esta hace justo lo contrario ampliando el foco sobre un único piloto y vuelta para diseccionar cómo condujo. Todos los módulos de esta categoría parten de la telemetría de una vuelta de referencia, normalmente, la más rápida del piloto obtenida con el patrón de acceso descrito en la sección 4.2. Estos datos se presentan de tres

formas complementarias: como cifras de rendimiento, pintadas sobre el trazado del circuito, y como curvas de los distintos canales de telemetría. La categoría agrupa cuatro módulos desde un resumen del piloto hasta las trazas de telemetría.

4.4.1 Resumen del Piloto

El módulo `DriverReport.py` condensa el rendimiento de un piloto en una ficha. A partir de su vuelta más rápida extrae indicadores como velocidad punta y tiempos por sector. La Fórmula 1 divide cada circuito en tres sectores cronometrados de forma independiente, de modo que comparar sector a sector revela en qué parte del trazado un piloto gana o pierde tiempo. Estos tiempos se presentan con su diferencia a la mejor marca, como puede verse en la Figura 4.5.

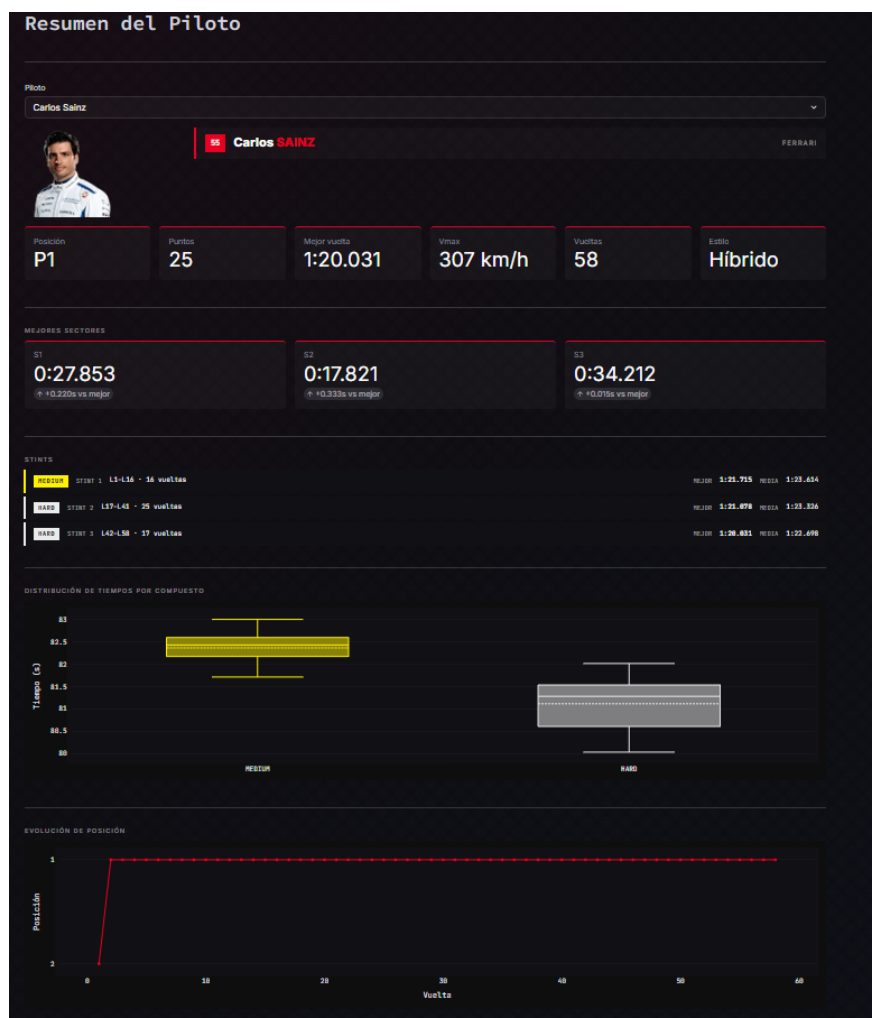


Figura 4.5. Vista de *Resumen del Piloto*: KPIs de la vuelta, tiempos por sector y distribución de la sesión.

4.4.2 Mapas sobre el trazado: velocidad y marchas

Los módulos `SpeedHeatMap.py` y `GearHeatMap.py` comparten una misma idea y se explican juntos por ello. Tratan de proyectar un canal de telemetría sobre la silueta del circuito. Como cada muestra de telemetría incluye la posición del coche X e Y como su estado en ese instante, se puede dibujar el trazado y colorear cada punto según el valor que tomaba allí una variable completa. Este estilo de dibujar la silueta del trazado está realizado de la misma forma que en el módulo `SessionSummary.py`, la documentación de FastF1 da una indicación sobre cómo dibujar el circuito.

En el **mapa de velocidad**, el color codifica la velocidad (canal `Speed`). De un solo vistazo se distinguen las rectas, donde el coche alcanza su máximo, de las curvas lentas, donde frena. En el **mapa de marchas**, el color muestra qué marcha usó el piloto en cada punto del trazado. Ambos permiten, opcionalmente, marcar las curvas numeradas para situarse mejor.

De forma común a ambos, la descarga de telemetría se separa del dibujado en una función cacheada con `@st.cache_data`, porque este decorador no puede envolver código que contenga llamadas a la interfaz de Streamlit. Así, cambiar una opción de visualización no obliga a volver a descargar los datos. La Figura 4.6 muestra un ejemplo de mapa de velocidad sobre el trazado.

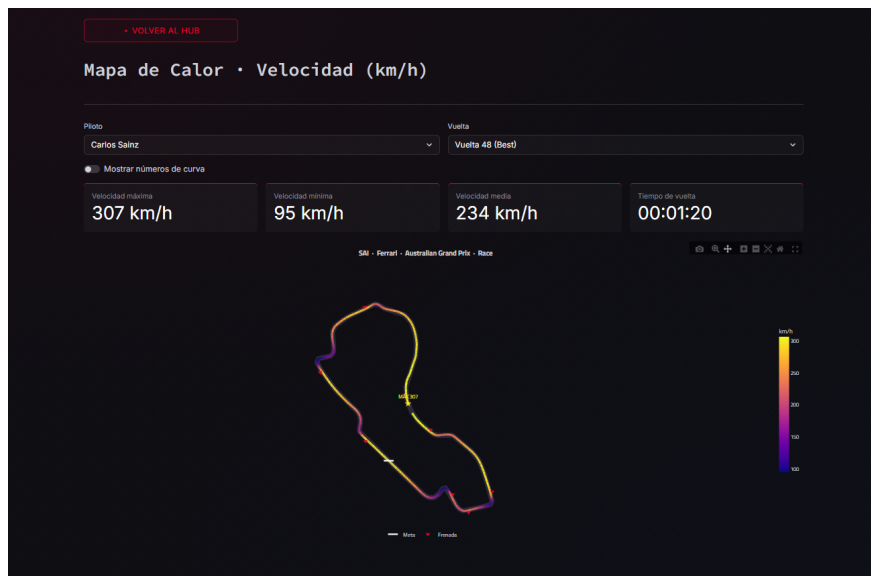


Figura 4.6. Mapa de velocidad: el trazado del circuito coloreado según la velocidad en cada punto. El mapa de marchas es análogo, codificando la marcha seleccionada.

4.4.3 Telemetry Trace

El módulo `Telemetrytrace.py` es el más completo de la categoría y el que da acceso al detalle más fino. En lugar de proyectar un único canal sobre el circuito, representa la evolución de todos los canales de telemetría a lo largo de la vuelta: velocidad, acelerador, freno, marcha, revoluciones del motor **RPM** y **DRS**, todos ellos frente a la distancia recorrida. Esto permite estudiar el estilo de conducción punto por punto, dónde levanta el pie, cuánto tiempo frena, cuándo abre el DRS.

El **DRS** (*Drag Reduction System*) es un alerón trasero móvil que el piloto puede abrir para reducir la resistencia aerodinámica y ganar velocidad punta en las rectas. No se puede usar libremente, solo en zonas designadas del circuito y únicamente cuando se circula a menos de un segundo del coche de delante, ya que su finalidad es facilitar los adelantamientos. En la telemetría, el canal **DRS** indica si el sistema está abierto (Su valor es mayor a ocho) o cerrado en cada punto (menor o igual que ocho). La Figura 4.7 ilustra el efecto del DRS sobre el flujo de aire, con el alerón cerrado el flujo se desordena tras el ala generando resistencia, mientras que con el DRS abierto atraviesa la ranura de forma más limpia, reduciendo el *drag*.

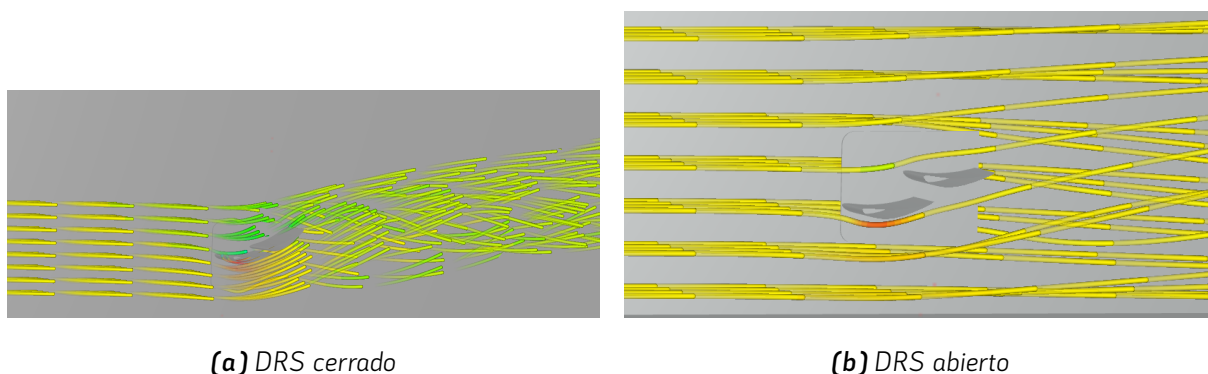


Figura 4.7. Simulación CFD ilustrativa del flujo de aire sobre el alerón trasero con el DRS cerrado y abierto. Imágenes propias realizadas sobre un modelo público de SimScale (autor: johmyang060323).

Se incluye únicamente para ilustrar el concepto; no forma parte del desarrollo ni se emplean sus resultados cuantitativos.

Conviene además mencionar que el canal de freno **Brake** se entrega en FastF1 como **binario**, indica únicamente si el piloto está frenando o no sin registrar la presión real sobre el pedal. Por ello lo que la herramienta representa como freno es un estado de apagado o encendido. La Figura 4.8 recoge esta vista con todos los canales.



Figura 4.8. Vista de Telemetry Trace: evolución de los canales de telemetría (velocidad, acelerador, freno, marcha, RPM y DRS) a lo largo de la vuelta frente a la distancia.

Este módulo es también el que integra los dos clasificadores de piloto: el tipo de vuelta y la firma de pilotaje. Ambos modelos construidos y validados en la sección 4.7. La Figura 4.9 muestra ambos clasificadores en funcionamiento.

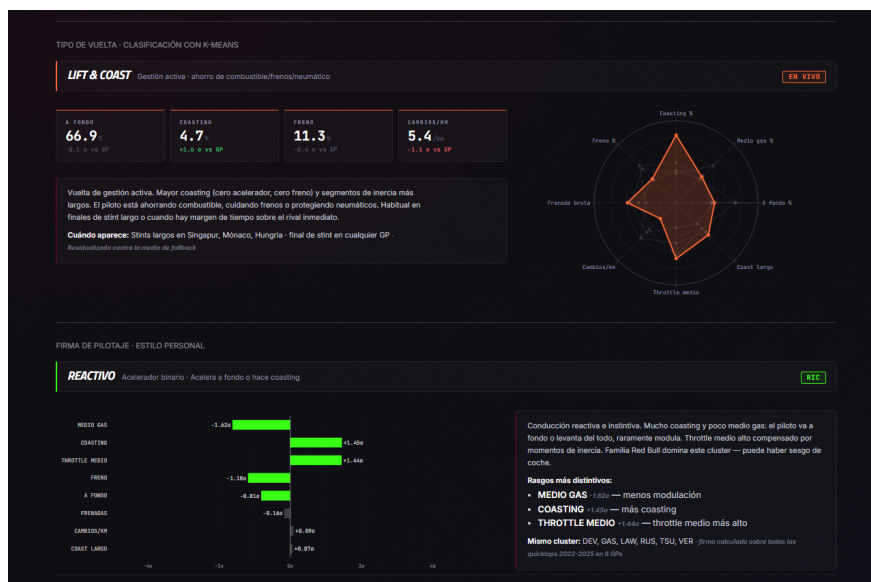


Figura 4.9. Los dos clasificadores en acción sobre una vuelta de Daniel Ricciardo (GP de Australia 2024): arriba el tipo de vuelta (Lift & Coast, gestión) y abajo la firma de pilotaje (Reactivo), ambos calculados en vivo. Su construcción se detalla en la Sección 4.7.

4.4.3.1 Circuito animado

Integrada dentro de la traza de telemetría, la herramienta ofrece una simulación animada de la vuelta. `Circuit2d.py`: un punto recorre el trazado a la vez que se desplazan unos paneles con la evolución de los pedales, marcando además las zonas de DRS y los tiempos por sector con el código de colores habitual de la retransmisión (morado para el mejor tiempo absoluto, verde para el mejor personal, amarillo para el resto).

Este módulo es un ejemplo de decisión motivada por el rendimiento. En su primera iteración la animación se generaba con *Matplotlib*, renderizándola como vídeo que se descargaba e incrustaba como HTML en la página. El enfoque funcionaba, pero era costoso de generar y lento de cargar. Se reimplementó por completo de forma nativa en *Plotly*, como una animación interactiva sin paso intermedio por vídeo, eliminando así la descarga y reduciendo el tiempo de carga. La Figura 4.10 muestra un fotograma de esta simulación.



Figura 4.10. Simulación animada de la vuelta: un punto recorre el trazado mientras se desplazan los paneles de pedales y se marcan zonas de DRS y tiempos por sector.

4.5 Análisis comparativo

Las dos primeras categorías analizan a un piloto o un evento de forma aislada. Esta tercera se dedica a enfrentar a dos o más pilotos entre sí, que es donde el análisis de telemetría revela quién es más rápido y, sobre todo, dónde. Toda la categoría comparte un reto técnico común.

La telemetría de dos vueltas no es directamente comparable: cada una contiene sus muestras en puntos distintos del trazado, porque dependen del tiempo y no de la posición. Superponerlas en crudo no tiene sentido. La solución, común a todos los módulos de esta categoría, es interpolar ambas vueltas sobre una rejilla de distancia común. Se define una malla de puntos igualmente espaciados a lo largo del trazado y se muestrea cada canal de cada vuelta sobre esa malla. Una vez ambas vueltas comparten el mismo eje distancia, sus puntos corresponden uno a uno y la comparación es legítima. Sobre esa base se calcula el *delta*: la diferencia de tiempo acumulada entre las dos vueltas en cada punto del circuito, que indica cuánto tiempo gana o pierde un piloto respecto al otro a lo largo de la vuelta. El listado 4.6 muestra cómo se interpolan dichos canales.

```
1 def _interpolate_on_grid(tel, dist_grid, ref_dist_max):
2     src_dist = tel["Distance"].values.astype(float)
3     # Normalizar por fracción de vuelta (robusto a trazadas de distinta
4         longitud)
5     frac_orig = src_dist / src_dist[-1]
6     frac_grid = dist_grid / ref_dist_max
7
8     # Remuestrear cada canal sobre la rejilla común
9     speed = np.interp(frac_grid, frac_orig, tel["Speed"].values)
10
11     # Tiempo acumulado (clave para el delta), forzado a empezar en 0
12     time_s = tel["Time"].dt.total_seconds().values
13     time_s = np.interp(frac_grid, frac_orig, time_s - time_s[0])
14     # ... (resto de canales: throttle, brake, gear, rpm, drs)
15     return {"speed": speed, "time_s": time_s, ...}
```

Listado 4.6. Interpolación de una vuelta sobre la rejilla común en *Ghostcar.py*.

4.5.1 Panel de Equipo

El módulo `TeamTelemetry.py` enfrenta a los dos pilotos de un mismo equipo. Estos dos pilotos comparten monoplace y son por tanto la comparación más limpia posible. Se muestra su telemetría de forma simétrica con una vista inspirada en las interfaces de telemetría que los propios equipos usan en el muro de boxes durante la retransmisión. Se añade en el centro el circuito dibujado indicando en una misma vuelta donde se encontraban físicamente dichos pilotos.

4.5.2 Comparativa de sesión

El módulo `OverallStandings.py` superpone los canales de telemetría de varios pilotos sobre una misma vuelta, se interpolan sobre la rejilla común permitiendo de esta manera contrastar los mismos canales de telemetría que encontrábamos en `Telemetrytrace.py`. La herramienta permite seleccionar hasta cuatro pilotos para superponer su telemetría. Este módulo cuenta también con una sección *Head 2 Head* que permite ver el dominio de un piloto sobre otro de forma visual viéndolo sobre el circuito pintado. También nos permite mirar el delta acumulado de las dos vueltas seleccionadas. Por último permite ver un desglose de los *stints* de los pilotos seleccionados viendo la evolución del compuesto por vuelta y una tabla resumen de los *stints*.

4.5.3 La vuelta ideal

El módulo de `OptimalLap.py` construye una vuelta teórica perfecta, que ningún piloto llega a dar. Una vuelta teórica se construye dividiendo el circuito en un número a seleccionar de *microsectores* (Pequeños tramos consecutivos). Para cada microsector identifica qué piloto y equipo fue el más rápido en él. La vuelta ideal es el mosaico resultante con cada pequeño sector coloreado con el equipo dominante y su tiempo es la suma de los mejores tiempos parciales de toda la parrilla. Representa el límite absoluto de rendimiento alcanzable en ese circuito combinando lo mejor de cada coche. El módulo permite filtrar por equipo para solo ver los resultados de determinadas escuderías. La Figura 4.11 muestra esta vuelta ideal por microsectores.

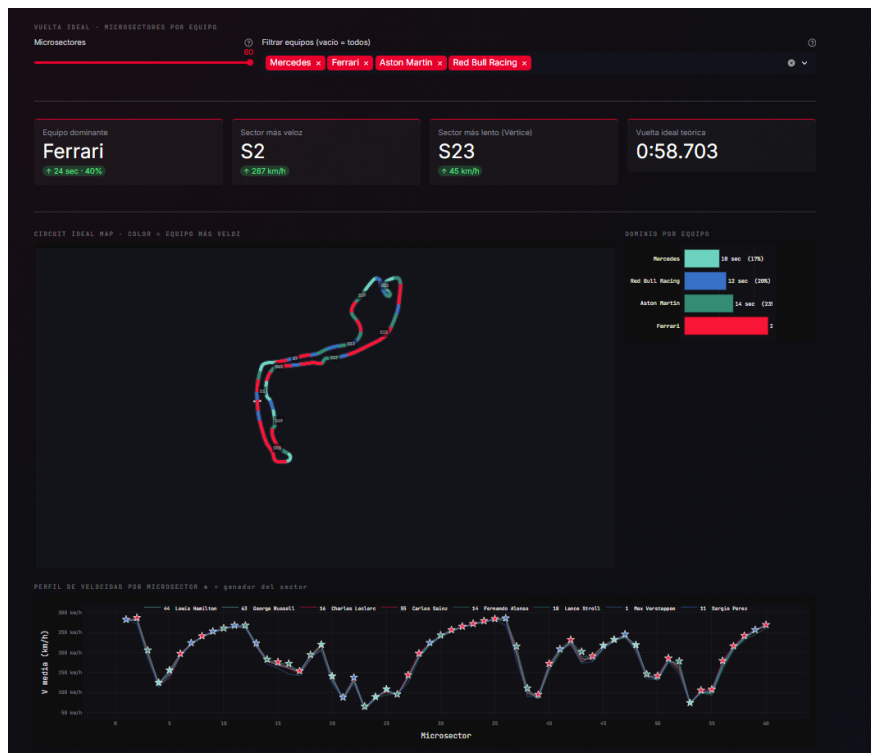


Figura 4.11. La Vuelta Ideal: el circuito dividido en microsectores, cada uno coloreado con el equipo más rápido en él y filtrado por equipos. El tiempo resultante es la suma de los mejores parciales de toda la parrilla.

4.5.4 Coche fantasma

El módulo `Ghostcar.py` lleva la comparación al plano visual. Tras interpolar dos vueltas sobre la rejilla común, las reproduce como una **animación de persecución** donde dos coches recorren el trazado simultáneamente y, al ir cada uno a su ritmo real, el más rápido se distancia progresivamente del otro. El nombre alude a la mecánica del *coche fantasma* de los videojuegos de conducción. La animación se acompaña del gráfico de *delta*, de modo que la diferencia de tiempo no solo se ve crecer entre los coches sino que se cuantifica en cada punto de la vuelta. La Figura 4.12 recoge un ejemplo de esta comparación.



Figura 4.12. *Coche Fantasma (Qualy GP de Mónaco 2023): dos vueltas interpoladas sobre la rejilla común se reproducen como persecución; abajo, la curva del delta acumulado entre ambas.*

4.6 Dinámica vehicular

La última de las cuatro categorías es la más física de todas. Mientras las anteriores presentan o comparan canales de telemetría, esta los transforma en magnitudes físicas que no vienen medidas en los datos. Las fuerzas que actúan sobre el monoplaza son el núcleo del que parten los tres módulos de esta categoría.

4.6.1 Fundamento físico

Una fuerza G es una forma intuitiva de expresar una aceleración. Es el número de veces que esta fuerza supera a la aceleración de la gravedad ($g = 9,81 \text{ m/s}^2$). Cuando se dice que un piloto soporta 5G en una frenada, significa que su cuerpo experimenta una fuerza equivalente a cinco veces su peso. Es la magnitud que mejor comunica lo extremo del entorno físico de la Fórmula 1.

Sobre el coche actúan dos componentes independientes. La **aceleración longitudinal** actúa en la dirección del movimiento, es positiva al acelerar y negativa al frenar. La **aceleración lateral** actúa

perpendicular a la marcha y aparece en las curvas, empujando el coche hacia el exterior del giro. Las limitaciones técnicas son claras puesto que FastF1 no entrega aceleraciones. Entrega la posición del coche y su velocidad escalar. Las fuerzas G hay que **derivarlas** de esos datos.

Aceleración longitudinal.

Es la más directa. La aceleración es, por definición, la variación de la velocidad con el tiempo:

$$a_{\text{lon}} = \frac{dv}{dt}, \quad G_{\text{lon}} = \frac{a_{\text{lon}}}{g}. \quad (4.1)$$

Como la velocidad v está disponible, basta con derivarla respecto al tiempo.

Aceleración lateral.

Es más sutil, porque no puede obtenerse de la velocidad escalar. Un coche puede ir a velocidad constante y aun así sufrir una gran aceleración lateral si gira. Aquí entra la física del movimiento circular. Un coche que toma una curva de radio R describe un arco, y para no salirse en línea recta necesita una **aceleración centrípeta** dirigida hacia el centro de la curva:

$$a_{\text{lat}} = \frac{v^2}{R} = v \cdot \omega, \quad (4.2)$$

donde $\omega = v/R$ es la **velocidad angular** o ritmo de giro: cuántos radianes por segundo cambia la dirección de marcha del coche. Esta dirección, el *heading* θ , se obtiene de cómo varía la posición:

$$\theta = \arctan2\left(\frac{dY}{dt}, \frac{dX}{dt}\right), \quad \omega = \frac{d\theta}{dt}. \quad (4.3)$$

Es decir, se mira hacia dónde apunta la trayectoria en cada instante y se calcula la rapidez con la que ese ángulo cambia. Multiplicando por la velocidad se obtiene la aceleración lateral.

Uso de la teoría

Las fórmulas son exactas, pero aplicarlas sobre datos GPS muestreados tiene un problema. Al derivar dos veces la posición se **amplifica el ruido**. Por ello el cálculo añade tres protecciones mínimas, sin alterar la física. Se añade un filtro de tipo Savitzky-Golay (toda derivada numérica de datos discretos lo requiere), la anulación de la G lateral por debajo de 30 km/h donde la dirección de marcha no es fiable, como en el *pit lane* o las vueltas de entrada y salida y un recorte final al límite físico de $\pm 6G$ como red de seguridad. El listado siguiente recoge el cálculo completo.

Savitzky-Golay

Derivar numéricamente una señal muestreada amplifica el ruido de alta frecuencia, por lo que antes de derivar se suaviza con un filtro Savitzky-Golay [27, 28]. A diferencia de una media móvil que aplanan los picos, este filtro ajusta por mínimos cuadrados un polinomio de grado bajo a una ventana deslizante $2m + 1$ muestras y evalúa ese polinomio en el punto central.

$$y_i^{\text{suav}} = \sum_{j=-m}^m c_j y_{i+j}, \quad (4.4)$$

donde los coeficientes c_j se obtienen del ajuste polinómico y solo dependen del tamaño de la ventana y el grado del polinomio. De este modo se reduce el ruido preservando la forma de los picos de aceleración, que es justo lo que interesa conservar. La implementación emplea la función `savgol_filter` de SciPy con una ventana equivalente a medio segundo y un polinomio de grado tres. Podemos observar en el listado 4.7 como se calculan las fuerzas y cómo se usa el filtro mencionado.

```
1 # G longitudinal: derivada de la velocidad
2 g_lon = savgol_filter(np.gradient(speed_ms, time_s) / 9.81, win, poly)
3
4 # G lateral: v * omega, con omega = ritmo de giro de la dirección
5 x = savgol_filter(tel["X"].values, win, poly)
6 y = savgol_filter(tel["Y"].values, win, poly)
7 heading = np.unwrap(np.arctan2(np.gradient(y, time_s),
8                               np.gradient(x, time_s)))
9 omega = np.gradient(heading, time_s)
10 g_lat = savgol_filter(speed_ms * omega / 9.81, win, poly)
11 g_lat[speed_ms < 30/3.6] = 0.0 # dirección no fiable a baja velocidad
12
13 g_total = np.sqrt(g_lat**2 + g_lon**2)
```

Listado 4.7. Cálculo de las fuerzas G en *GGDiagram.py*.

Validación del método

El cálculo longitudinal es sólido, aplicado a vueltas de clasificación produce frenadas pico del orden de 5G que está en línea con las cifras públicas de la categoría. El cálculo lateral, en cambio, es una **estimación cinemática a partir del GPS**, no una medida de una unidad inercial real. La estimación de

la dinámica de un vehículo de competición a partir de sensores de posición y velocidad es un problema estudiado en la literatura [33]. Captura fielmente la forma de las cargas en curva, pero el ruido de la señal de posición tiende a sobreestimar los picos, que se acotan al límite físico. Se documenta así siendo un estimador, no una medición instrumentada.

4.6.2 El Diagrama G-G

El módulo `GGDiagram.py` representa cada instante de la vuelta como un punto en un plano cuyos ejes son la G lateral y la G longitudinal. El resultado es el llamado **círculo de tracción** o círculo de fricción, un concepto clásico de la dinámica del neumático [23]. La idea física es que un neumático dispone de un cierto límite de agarre al que el piloto puede aproximarse frenando, girando o una combinación de ambos, pero cuya suma no puede superar cierto máximo o el coche se descontrolará. La envolvente convexa de todos los puntos dibuja ese límite, y su forma revela el estilo de pilotaje: un buen piloto mantiene los puntos pegados al borde, exprimiendo el agarre disponible en todo momento.

4.6.3 Carga Aerodinámica

La aerodinámica es uno de los pilares del rendimiento en Fórmula 1 [14]. Al avanzar, el coche genera **carga aerodinámica** (*downforce*). Una fuerza vertical que lo presiona contra el asfalto y aumenta el agarre, a costa de una resistencia al avance (*drag*). Ambas fuerzas comparten la misma forma física y crecen con el **cuadrado de la velocidad**, diferenciándose únicamente en el coeficiente aerodinámico:

$$L = \frac{1}{2} \rho v^2 C_l A, \quad D = \frac{1}{2} \rho v^2 C_d A, \quad (4.5)$$

donde L es la carga aerodinámica (*downforce*) y D la resistencia (*drag*); ρ es la densidad del aire, v la velocidad, A el área frontal y C_l, C_d los coeficientes de sustentación y resistencia respectivamente. Esta dependencia con v^2 es la clave que permite detectar la carga aerodinámica a partir de la telemetría.

El razonamiento del módulo `Aero.py` es el siguiente. Sin aerodinámica, el agarre lateral máximo de un coche sería constante, fijado solo por el agarre mecánico (basado en neumáticos y peso). Pero con *downforce*, ese agarre crece con la velocidad, porque la carga vertical aumenta con la velocidad al cuadrado. El máximo agarre lateral se modela entonces como:

$$G_{\text{lat,max}}(v) = \alpha + \beta v^2, \quad (4.6)$$

donde α representa el agarre mecánico base y β la ganancia por carga aerodinámica. Si al ajustar este

modelo resulta que $\beta > 0$ con significación estadística, se tiene evidencia de la firma del *downforce*. El módulo agrupa los datos por velocidad, calcula la envolvente de agarre lateral en cada tramo y realiza una regresión por mínimos cuadrados de la envolvente frente a v^2 , reportando la pendiente β , el coeficiente de determinación R^2 y el p -valor.

En cuanto al alcance del módulo, este **detecta la firma** de la carga aerodinámica de forma relativa, no la mide en Newtons. Los parámetros α y β son compuestos puesto que no separan masa, área ni coeficientes y solo son interpretables comparativamente entre pilotos y sesiones. Es una estimación indirecta a partir de la telemetría, sin instrumentación física. La simulación de la Figura 4.13 se incluye para ilustrar el concepto.

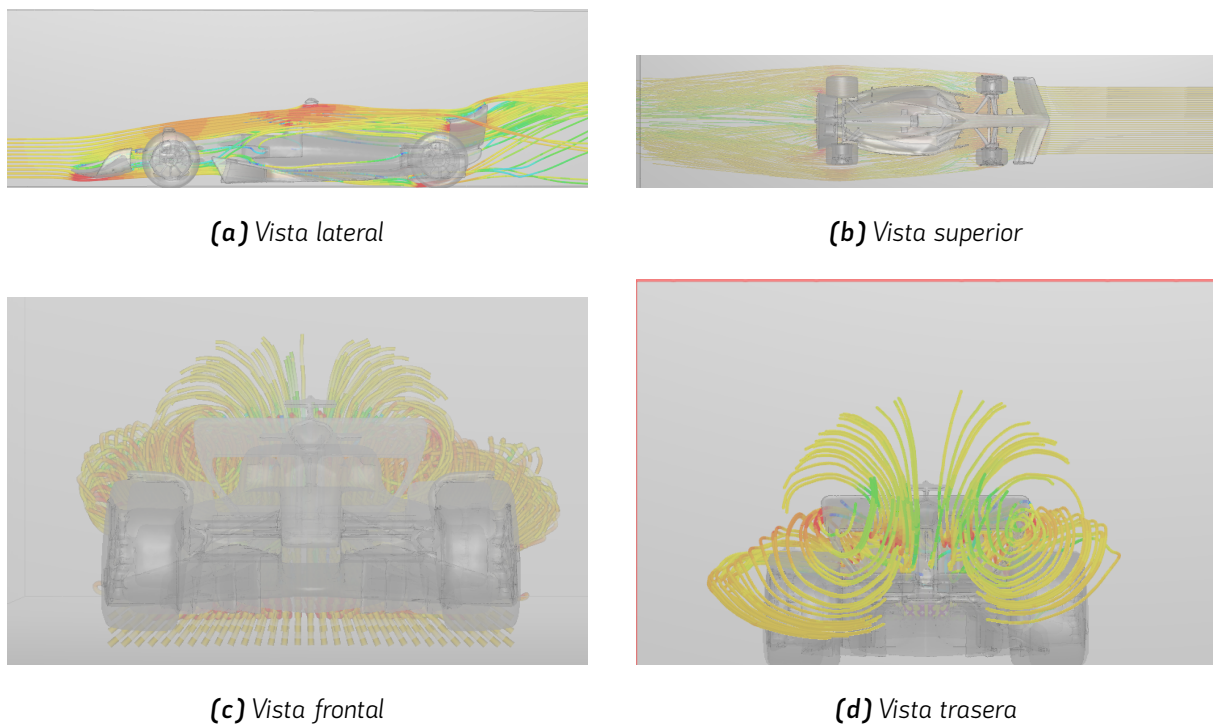


Figura 4.13. Simulación CFD ilustrativa del flujo de aire alrededor de un monoplaza, coloreada por magnitud de velocidad. Se aprecian la estela tras el coche y los vórtices de borde, fenómenos centrales en la aerodinámica de un Fórmula 1 [10]. Imágenes propias realizadas sobre un modelo público en SimScale (autor del modelo: alex_gugan). Se incluye únicamente para ilustrar el concepto de carga aerodinámica; no forma parte del desarrollo ni se emplean sus resultados cuantitativos.

4.6.4 Carga por rueda

El último módulo, `TyreLoad.py`, estima cómo se reparte la carga entre las cuatro ruedas a lo largo de la vuelta. La base física es la **transferencia de masas**. Cuando el coche frena, el peso se desplaza hacia

el eje delantero, cuando toma una curva se desplaza hacia las ruedas exteriores. Estas transferencias son proporcionales a las fuerzas G calculadas anteriormente y a la geometría del coche, donde h es la altura del centro de gravedad, L la distancia entre ejes y t el ancho de vía:

$$\Delta F_{z,\text{lon}} \propto G_{\text{lon}} \cdot \frac{h}{L}, \quad \Delta F_{z,\text{lat}} \propto G_{\text{lat}} \cdot \frac{h}{t}. \quad (4.7)$$

A la carga estática se le suman estas transferencias y la carga aerodinámica. Dicha carga crece con la velocidad y se reduce al activar el DRS. Comparando la fuerza que cada neumático debe transmitir con la que es capaz de soportar se obtiene su *exigencia*, una medida de cuánto se está estresando cada goma. La Figura 4.14 muestra esta representación de la carga por rueda.

El modelo es físicamente correcto en la formulación puesto que las leyes de transferencia de masas son las estándar de la dinámica vehicular [9, 18], si bien los parámetros del vehículo son valores realistas asumidos y no medidos del coche concreto. Es, por tanto, una estimación coherente y físicamente fundamentada y no una medición exacta por falta de datos.

4.7 Modelos de aprendizaje no supervisado

Tres de los módulos de la herramienta no se limitan a mostrar datos, sino que clasifican asignando una etiqueta interpretable a partir de la telemetría. Estos tres clasificadores comparten la misma arquitectura de aprendizaje automático, descrita de forma unificada en esta sección.

4.7.1 El problema y el enfoque no supervisado

El objetivo es agrupar miles de observaciones (vueltas, circuitos o pilotos) en familias de comportamiento similar, sin disponer de etiquetas previas. Nadie ha marcado de antemano qué vuelta es de *ataque*, *gestión* o qué circuito es *urbano*. Este es un problema de **aprendizaje no supervisado**, en el que el algoritmo descubre la estructura de datos por sí mismo. La técnica empleada es **K-Means**, uno de los algoritmos más establecidos.

4.7.2 Representación de las características

Un algoritmo de agrupamiento no entiende de etiquetas como vueltas o circuitos, solo de números. Por ello, cada observación se resume en un *vector de características*, una lista de valores numéricos que la

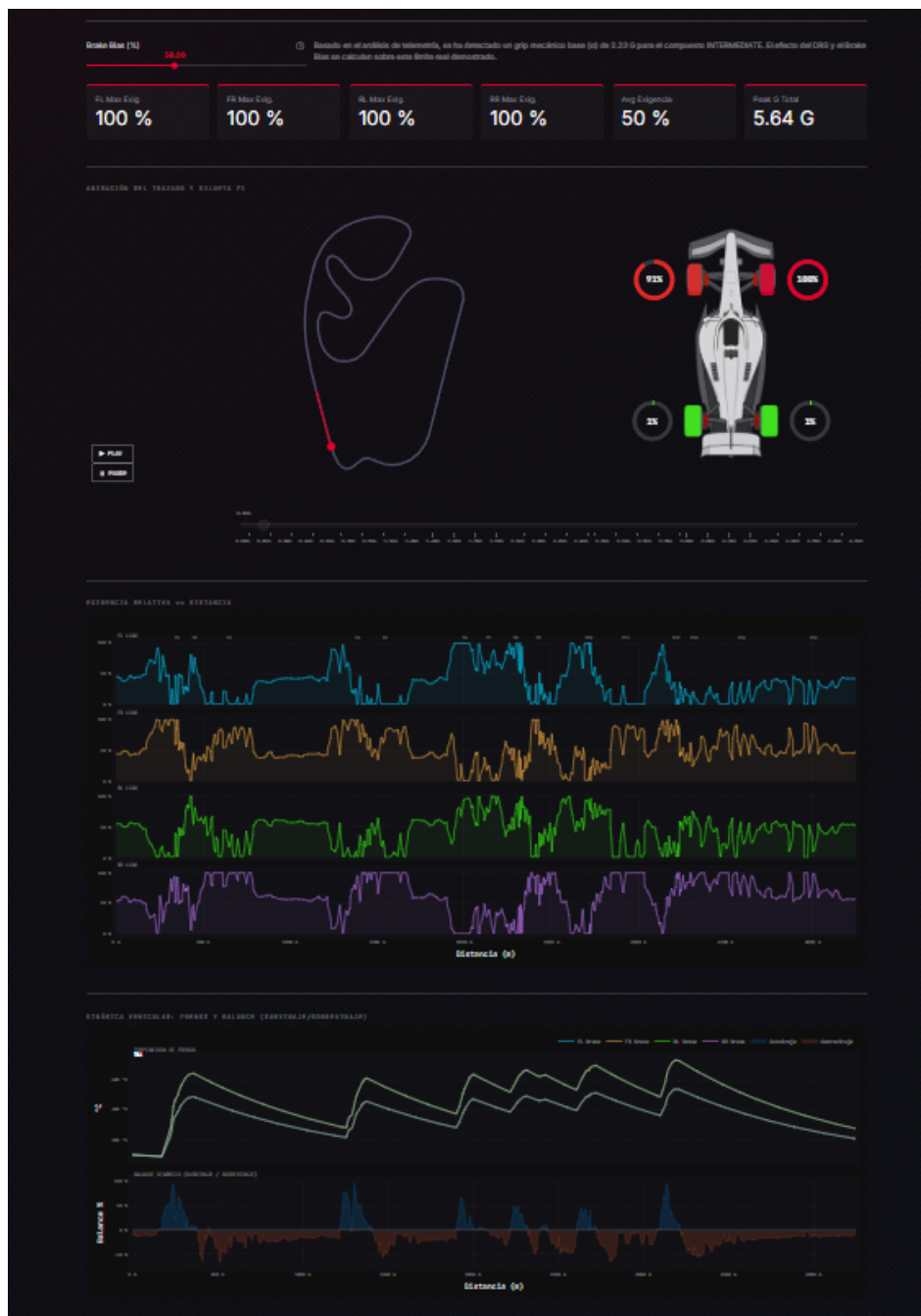


Figura 4.14. Mapa de Carga por Rueda: estimación de la carga vertical y la exigencia sobre cada neumático a lo largo de la vuelta, derivada de las transferencias de masa. Curva número 1 de brasil, frenada fuerte.

describen. Una vuelta, por ejemplo, se representa con ocho características de pilotaje (porcentaje de tiempo a pleno gas, frenada, inercia o *coasting*, cambios de marcha por kilómetro, entre otras), mientras que un circuito se describe mediante sus fuerzas G. Cada observación se convierte así en un punto dentro de un espacio de tantas dimensiones como *features* tenga.

4.7.3 Cómo agrupa K-Means

K-Means divide los puntos en k grupos buscando minimizar la distancia de cada punto al centro del grupo. El proceso es iterativo. Se colocan k centros iniciales, a cada punto se le asigna el más cercano. Cada centro se recoloca en la media de los puntos que se le han asignado y se repite hasta que los centros se estabilizan. El resultado por tanto serán k **centroides**, cada uno representando el punto medio de una familia. Una observación nueva se clasifica calculando a qué centroide está más cerca.

4.7.4 Refinamiento sobre el modelo base

El enfoque básico se mejora con dos técnicas que resultaron decisivas para la calidad de la clasificación.

Residualización por circuito

En el clasificador de tipo de vuelta, comparar valores crudos sería engañoso. Una vuelta en Mónaco siempre tendrá poco tiempo a pleno gas por la naturaleza del circuito, no por el estilo del piloto o por el tipo de vuelta. Para aislar el comportamiento del piloto del efecto del trazado, a cada vuelta se le resta la media de su Gran Premio. De este modo las características no miden cuánto acelera el piloto, sino cuánto se desvía de lo normal en ese circuito, haciendo comparables vueltas de circuitos distintos.

Reducción de dimensionalidad con PCA

En el clasificador de estilo de pilotaje, las ocho características están muy correlacionadas entre sí. El porcentaje a pleno gas y la media del acelerador, por ejemplo, miden cosas parecidas. Esta redundancia dispersa los grupos y dificulta la separación. Para resolverlo se aplica **Análisis de Componentes Principales** (PCA), una técnica que reescribe los datos en un número menor de dimensiones, las que capturan la mayor variación, descartando la redundancia. Proyectando las ocho características sobre sus dos componentes principales, que conservan en torno al 65% de la información, la separación entre grupos prácticamente se duplicó.

Entrenamiento offline e inferencia en vivo

El cálculo de los centroides es costoso, pero solo necesita realizarse una vez. Por ello el entrenamiento se ejecuta de forma aislada en los *scripts* en la carpeta reservada a investigación, apoyados en la implementación de K-Means y PCA de scikit-learn. Los centroides resultantes se almacenan como constantes en los módulos de producción. En tiempo de ejecución la aplicación no reentrena nada. Simplemente extrae las características de la observación, las compara con los centroides ya calculados y devuelve la etiqueta más próxima de forma instantánea. El preprocesado previo al entrenamiento difiere según el clasificador. El de tipo de vuelta, que opera sobre vueltas individuales, depura el conjunto con un *Isolation Forest* que aísla y descarta en torno al 5% de observaciones anómalas (vueltas tras coche de seguridad, de entrada y salida de boxes o con telemetría corrupta) antes de calcular los centroides. Los clasificadores de pilotaje y circuito no lo requieren, ya que trabajan sobre datos agregados (la media de las vueltas de cada piloto o de cada circuito), y esa agregación diluye por sí misma el efecto de las observaciones atípicas.

Observaciones atípicas

Si una observación queda demasiado lejos de todos los centroides, más allá de un umbral, no se le fuerza una etiqueta, sino que se marca como *atípica* o *híbrido* en el caso de pilotos. Esto es deliberado, ya que es preferible admitir que un perfil no encaja limpiamente a inventar una clasificación falsa. La validación cuantitativa de los clasificadores se presentará en el capítulo 5.

5

Validación y Pruebas

La validación de este trabajo se aborda en tres niveles complementarios, que cubren desde la corrección de cálculos internos hasta el comportamiento de cara al usuario. En primer lugar, una validación funcional comprueba que la herramienta reproduce hechos conocidos y verificables. En segundo lugar, una batería de pruebas unitarias verifica de forma automática la corrección de las funciones de cálculo. Por último, los modelos de aprendizaje no supervisado se evalúan con métricas específicas. Esta estructura sigue la pirámide habitual de validación desde la unidad más pequeña a la aceptación del sistema como conjunto.

5.1 Validación funcional

El objetivo de esta primera validación es comprobar que la información que presenta la herramienta se corresponde con la realidad de cada sesión. Para ello no basta con inspeccionar las pantallas sino que cada salida se contrasta con una fuente externa y verificable, el resultado oficial de la prueba. Se ha seleccionado un caso representativo de cada categoría funcional de la herramienta, recogiendo los resultados en la tabla [5.1](#).

Módulo	Caso comprobado	Esperado (fuente)	Obtenido	
Resumen de sesión	Ganador del GP de Baréin 2024	Verstappen (oficial F1)	Verstappen	✓
Cronología	Bandera roja en Mónaco 2024	Vuelta 1 (accidente)	Vuelta 1	✓
Análisis de piloto	Velocidad máxima de Verstappen, Mónaco 2023	≈290 km/h (circuito lento)	290 km/h	✓
Análisis de piloto	Marcha máxima en Mónaco	7ª (no se usa la 8ª)	7ª	✓
Comparativas	Diferencia Verstappen–Alonso en la pole de Mónaco 2023	+0,084 s (oficial F1)	+0,084 s	✓
Dinámica vehicular	Velocidad punta en Monza 2024	≈340 km/h (circuito más rápido)	339 km/h	✓

Tabla 5.1. Casos de validación funcional, uno por categoría de la herramienta, contrastados contra fuentes oficiales y conocimiento del dominio.

En todos los casos la salida de la herramienta coincide con la referencia, lo que confirma que tanto la lectura y presentación de datos como el cálculo de magnitudes derivadas se comportan de forma correcta sobre sesiones reales.

5.2 Pruebas Unitarias

Mientras que la validación funcional comprueba el sistema completo, las pruebas unitarias verifican de forma automática y aislada la corrección de las funciones de cálculo críticas, aquellas cuyo resultado es numérico y verificable. Se ha empleado *pytest*, el *framework* de pruebas estándar de Python.

5.2.1 Estrategia: telemetría sintética frente a *mocking*

El principal obstáculo para testear este código es que opera sobre datos descargados de FastF1, lo que introduce dependencia de la red y resultados no deterministas. La alternativa habitual sería *mockear* FastF1, es decir, sustituirlo por un objeto falso que imite su interfaz, sin embargo, la API de FastF1

es extensa y su imitación fiel resulta frágil y costosa de mantener. Se optó por una estrategia más robusta, construir directamente la telemetría sintética que las funciones de cálculo consumen (Un *DataFrame* con las columnas necesarias) generada a partir de física conocida. De este modo cada prueba dispone de un resultado esperado calculado de forma analítica con el que contrastar la salida del código, sin red y de forma determinista.

5.2.2 Niveles de verificación

Las pruebas no son homogéneas, sino que se organizan en tres niveles de rigor. El primero es la verificación analítica, en la que la salida se compara contra una fórmula cerrada. Por ejemplo, generando la telemetría en trayectoria circular de un radio conocido a velocidad constante, la aceleración lateral teórica es exactamente v^2/r , de modo que la fuerza G calculada debe coincidir con ese valor. El segundo a nivel de comportamiento, que somete la función a una entrada con resultado evidente y por último a nivel de casos límite que comprueba que la función devuelve la estructura esperada y que ante datos incompletos o mal formados responde de forma controlada. La batería consta de 59 pruebas cubriendo las funciones de cálculo puro recogidas en la tabla 5.2.

Componente	Pruebas	Tipo de verificación
Cálculo de fuerzas G	10	Analítica (círculo, recta)
Clasificador de tipo de vuelta	10	Determinista + casos límite
Clasificador de circuito	9	Determinista (lookup)
Extracción de características	9	Comportamiento extremo
Interpolación y sincronización	8	Interpolación exacta
Clasificador de pilotaje	7	Coherencia sobre datos reales
Corrección por combustible	6	Analítica
Total	59	

Tabla 5.2. Cobertura de las pruebas unitarias por componente.

La totalidad de las pruebas se superan. Conviene precisar su alcance, acotado a la lógica del cálculo, que es la parte con resultado verificable. No se incluyen pruebas de integración con FastF1 ni de interfaz, esta dimensión queda cubierta por la validación funcional de la sección anterior.

Las 59 pruebas cubren las funciones de cálculo críticas, que presentan una cobertura del 100%. La cobertura de líneas sobre el conjunto de los módulos es del 31%, ya que estos incluyen también el

código de la interfaz, que no es susceptible de prueba unitaria y queda fuera del alcance por diseño.

5.3 Validación de los clasificadores

Los tres modelos de aprendizaje no supervisado descritos en la sección 4.7 requieren una validación distinta a la de dentro del sistema. Al no existir etiquetas verdaderas o una *ground truth* a la que acogerse, no es posible medir una exactitud en el sentido clásico. En su lugar, la calidad de un agrupamiento se evalúa con dos familias de métricas: las que miden calidad interna de los clústeres y las que miden su estabilidad.

5.3.1 Métricas empleadas

Se utilizan tres métricas internas y una de estabilidad.

- **Coeficiente de Silhouette** (rango $[-1, 1]$): para cada observación compara su cercanía a su propio clúster frente al más próximo. Valores altos indican grupos bien separados; valores cercanos a 0, observaciones en la frontera entre grupos.
- **Índice de Davies-Bouldin** (≥ 0 , menor es mejor): razón entre la dispersión interna de los clústeres y la distancia entre ellos.
- **Índice de Calinski-Harabász** (mayor es mejor): cociente entre la dispersión entre grupos y la dispersión dentro de los grupos.
- **Índice de Rand Ajustado** (ARI, rango $[-1, 1]$): mide la concordancia entre dos particiones. Se emplea para la estabilidad: si al reentrenar el modelo con un subconjunto de los datos las observaciones reciben la misma etiqueta, el ARI es alto, lo que indica un modelo reproducible y no fruto del azar.

5.3.2 Selección del número de clústeres

El número de clústeres k no se fija arbitrariamente, sino que se selecciona combinando el método del codo (evolución de la inercia) con el coeficiente de Silhouette. La figura 5.1 muestra este análisis para el clasificador de tipo de vuelta. La inercia decae suavemente, sin un codo pronunciado, lo cual

es coherente con la naturaleza continua de los datos de pilotaje. El coeficiente de Silhouette alcanza un máximo local en $k = 4$ que es el valor adoptado. Para los clasificadores de circuito y pilotaje, en cambio, el número de clústeres se fijó priorizando la interpretabilidad de los grupos: cinco tipologías de circuito y cuatro estilos de pilotaje con significado de dominio claro, manteniendo un silhouette competitivo, ya que valores de k superiores fragmentaban los datos en grupos sin interpretación útil o con muy pocas observaciones.

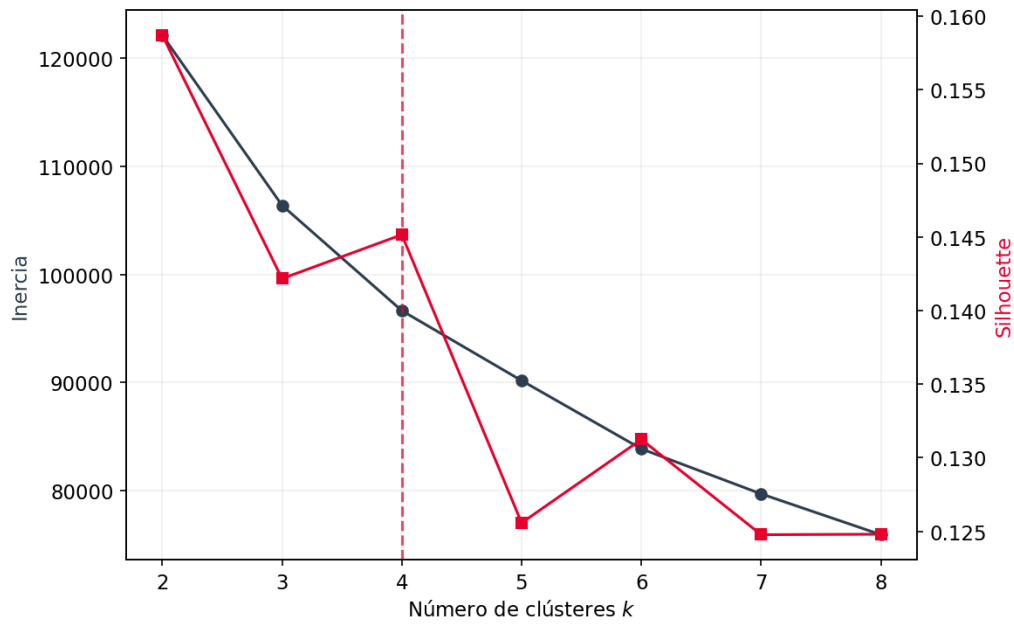


Figura 5.1. Selección de k para el clasificador de tipo de vuelta: inercia (codo) y coeficiente de Silhouette frente al número de clústeres. El máximo del Silhouette se alcanza en $k = 4$.

Este análisis se recoge en las figuras 5.2 y 5.3 para los clasificadores de pilotaje y de circuito, respectivamente.

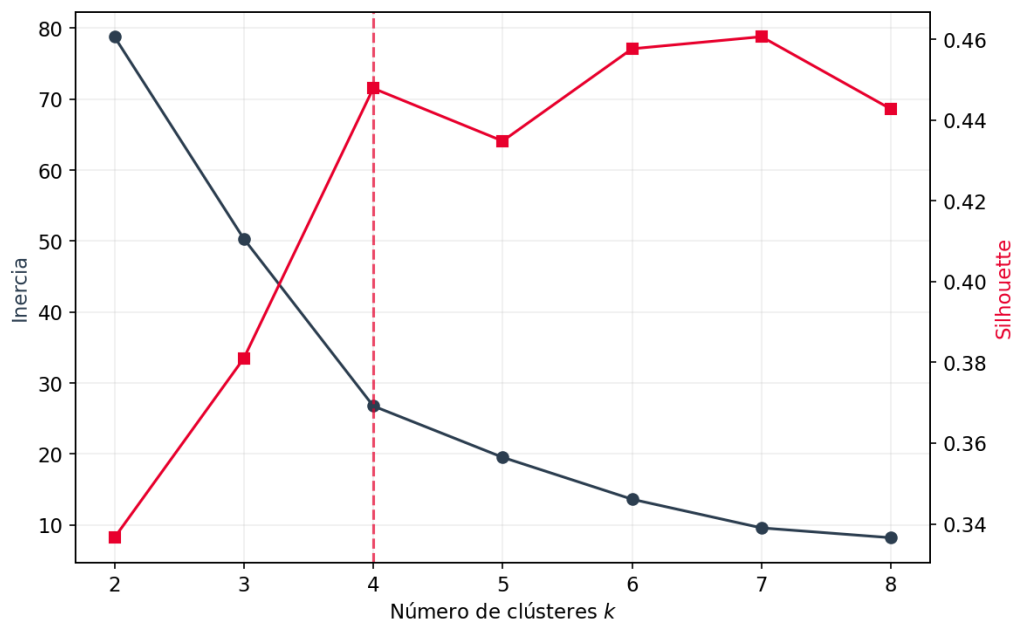


Figura 5.2. Selección de k para el clasificador de estilo de pilotaje. En $k = 4$ el silhouette es de 0,45, a solo 0,01 del máximo, pero con grupos interpretables y sin fragmentar en exceso a los pilotos.

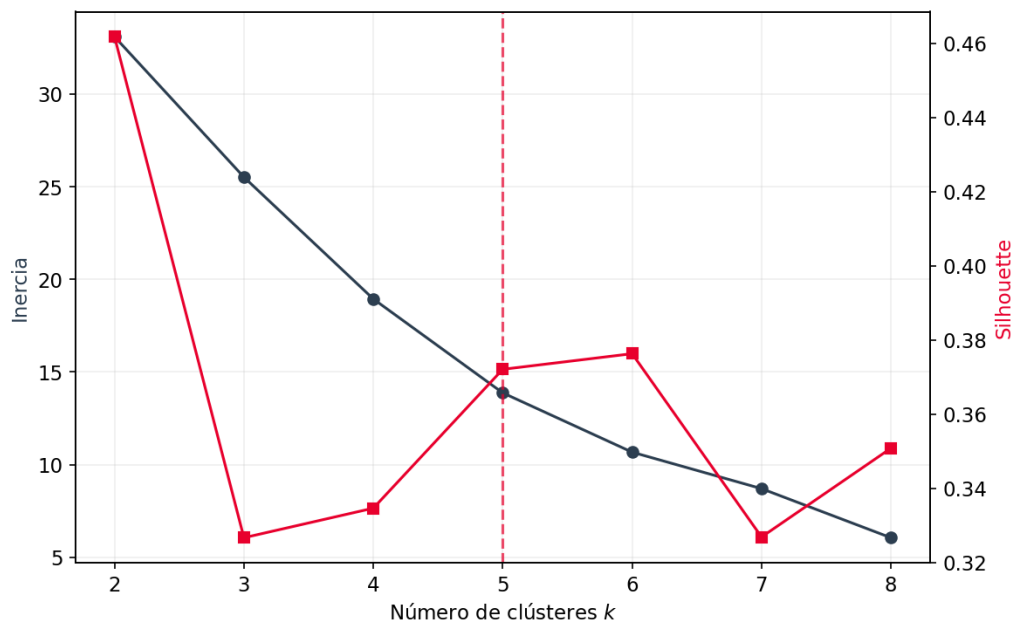


Figura 5.3. Selección de k para el clasificador de circuitos. Se adopta $k = 5$ por la interpretabilidad de las cinco tipologías resultantes, pese a que un k menor ofrezca un silhouette superior.

5.3.3 Resultados

La tabla 5.3 recoge las métricas de los tres clasificadores sobre el modelo desplegado, y las figuras 5.4 y 5.5 muestran la separación de los clústeres proyectada sobre las dos primeras componentes principales.

Clasificador	k	Silhouette	Davies-Bouldin	Calinski-H.	ARI
Pilotaje	4	0,45	0,65	26,8	1,00 (LOO)
Circuito	5	0,37	0,67	22,0	0,93 (LOO)
Tipo de vuelta	4	0,15	1,73	3007,8	0,98 (5-fold)

Tabla 5.3. Métricas de validación de los tres clasificadores. ARI obtenido por validación cruzada (leave-one-out para circuito y pilotaje, dado su menor tamaño, y 5-fold para tipo de vuelta).

Los resultados han de leerse con honestidad. Los clasificadores de pilotaje y de circuito presentan coeficientes de silhouette razonables e índices de Davies-Bouldin bajos, lo que indica que los grupos están bien definidos. El clasificador de tipo de vuelta presenta un Silhouette bajo, lo cual refleja la naturaleza continua de los datos.

Cabe aclarar que el índice de Calinski-Harabász depende del tamaño de la muestra, por lo que solo es comparable entre valores de k de un mismo clasificador, no entre distintos.

La proyección de los clústeres sobre sus dos primeras componentes principales confirma visualmente esta lectura. En los circuitos de la figura 5.4 los grupos aparecen claramente separados y Mónaco queda completamente aislado, coherente con su carácter de trazado Urbano Extremo.

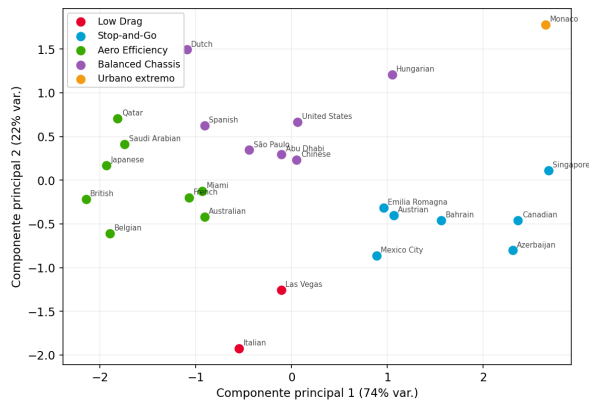


Figura 5.4. Clústeres de circuitos proyectados sobre sus dos componentes principales (95,8% de la varianza). Mónaco queda completamente aislado del resto, confirmando su carácter de trazado único.

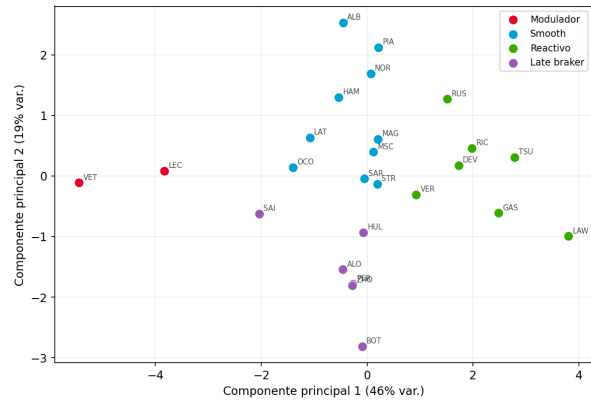


Figura 5.5. Clústeres de estilo de pilotaje proyectados sobre sus dos componentes principales (64,5% de la varianza), que sustentan la separación de los cuatro perfiles.

Los estilos de pilotaje de la figura 5.5 también se distinguen sobre sus componentes principales, sustentando la separación de los cuatro perfiles. En esto se puede ver cómo pilotos de la escudería Ferrari tienen un mismo estilo (posible sesgo de coche).

Como limitación principal, la estabilidad temporal del clasificador de pilotaje es baja entre temporadas distintas. Esto es esperable, ya que la firma de un piloto evoluciona con los cambios de coche y reglamento de cada año. No obstante, lo determinante es que, pese al solapamiento, las particiones son muy estables: el ARI alcanza valores de 0,93, lo que confirma que las asignaciones son reproducibles.

6

Conclusiones y Trabajos futuros

Este trabajo partió de una convicción simple: los datos de Fórmula 1 ya son públicos y accesibles, y lo que faltaba era una forma gratuita de explorarlos a fondo. Este proyecto materializa esa visión en una herramienta de análisis que cubre, en un único sistema integrado, gran parte del dominio.

A lo largo del desarrollo se han construido catorce módulos de análisis agrupados en cuatro categorías funcionales, tres clasificadores de aprendizaje no supervisado y un módulo de dinámica vehicular que deriva magnitudes de la telemetría. La arquitectura resultante es modular, reproducible e instalable en cualquier sistema Windows y Linux con Python sin dependencias de pago.

Desde el punto de vista técnico los principales retos abordados han sido el procesamiento de telemetría y muestreo irregular, las magnitudes físicas y el diseño de clasificadores no supervisados que ofrecen etiquetas interpretables, así como montar la aplicación funcional desde cero. La validación cuantitativa de los clasificadores afirma que, aunque hay una separación moderada por la naturaleza del dominio, la estabilidad es alta.

El desarrollo dejó también varias lecciones. La principal fue trabajar con datos reales que obligan a diseñar en torno a su imperfección, la telemetría llega con un muestreo irregular, dañada o con datos de ruido que obligó a procesarlas para poder compararlas. Las fuerzas G y las posiciones GPS exigieron un filtrado de ruido y en general, cada decisión técnica vino impuesta por una limitación concreta de los datos, no por una preferencia previa.

Otra lección surgió de los clasificadores. Fue el primer contacto con el aprendizaje no supervisado a un dominio real, y el principal reto no fue técnico sino conceptual al agrupar un fenómeno continuo que carece de fronteras nítidas. Elegir el número de grupos equilibrando las métricas con el significado

del dominio, y dar a cada clúster una etiqueta con sentido, enseñó que en este tipo de problemas la interpretabilidad pesa tanto como cualquier indicador numérico.

En cuanto al público objetivo, TALOS ha sido diseñado para ser útil en un espectro amplio de usuarios, desde el aficionado ocasional que quiere entender por qué su piloto favorito perdió la carrera hasta el entusiasta técnico que compara fuerzas G en trazados distintos. La ausencia de barreras de entrada (gratuito, sin registro, sin conocimientos de programación) es en sí misma parte del objetivo del proyecto.

6.1 Trabajo Futuro

El alcance de este trabajo deja abiertas varias líneas de mejora que se identificaron durante el desarrollo y la validación, y que representan extensiones naturales del sistema.

Clasificador de tipo de vuelta con pertenencia suave: El bajo Silhouette del clasificador refleja la naturaleza continua del estilo de conducción, una mejora conceptual sería sustituir k-means por un modelo distinto para modelar de una mejor manera el clasificador sin depender de fronteras discretas.

Integración de telemetría de simuladores: La arquitectura modular permite añadir nuevas fuentes de datos sin alterar el núcleo de la aplicación. Una extensión natural sería capturar los paquetes UDP que emiten videojuegos como *EA Sports F1* o simuladores como *Assetto Corsa Competizione*, permitiendo a los jugadores comparar su propia telemetría con la de pilotos reales.

Integración de competiciones: Aunque el deporte objetivo es la Fórmula 1, la herramienta funciona mediante datos que pueden provenir de cualquier fuente. Esto da pie a que, siempre que haya datos disponibles, pueda analizarse cualquier deporte de motor incluyendo competiciones de resistencia como el *World Endurance Championship* o categorías inferiores de Fórmula.

Independencia del proveedor de datos: Actualmente la aplicación depende directamente de FastF1 como fuente de telemetría. Una mejora estructural sería introducir una capa de abstracción entre la fuente de datos y la lógica de negocio, de modo que un cambio en la API de FastF1, o su eventual discontinuación, no afectara al resto del sistema y permitiera incorporar nuevas fuentes con un impacto mínimo.

Apéndice A. Manual de Instalación

El código fuente de TALOS está disponible públicamente en el repositorio:

<https://github.com/hgdario/f1-telemetry-dss>

A.1 Requisitos previos

- **Python 3.10 o superior.** Disponible en <https://www.python.org/downloads/>.
- **Conexión a Internet.** Necesaria durante la instalación para descargar las dependencias y, en la primera consulta de cada sesión, para que FastF1 descargue los datos desde los servidores de Fórmula 1. Las sesiones consultadas quedan almacenadas en caché local y no requieren conexión en accesos posteriores.
- **Git** (opcional). Necesario para clonar el repositorio; alternativamente puede descargarse el archivo ZIP desde la página del repositorio.

A.2 Obtención del código fuente

```
1 git clone https://github.com/hgdario/f1-telemetry-dss.git
2 cd f1-telemetry-dss
```

Listado 6.1. Clonado del repositorio

A.3 Instalación en Windows

En Windows la instalación y el arranque están automatizados mediante scripts .bat

1. Instalar Python 3.10 o superior desde <https://www.python.org/downloads/>, marcando la opción *Add Python to PATH* durante la instalación.
2. Ejecutar `install.bat` con doble clic. El script detecta Python, crea un entorno virtual aislado en `.venv` e instala todas las dependencias listadas en `requirements.txt`.
3. Una vez completada la instalación, ejecutar `start.bat` para arrancar la aplicación.

A.4 Instalación en Linux

En sistemas Unix el proceso equivalente se realiza desde el terminal. En distribuciones basadas en Debian puede ser necesario instalar previamente el módulo `venv`.

```
1 # Solo en Debian/Ubuntu/Mint, si el modulo venv no esta disponible
2 sudo apt install python3-venv
3
4 # Crear y activar el entorno virtual
5 python3 -m venv .venv
6 source .venv/bin/activate
7
8 # Instalar dependencias
9 pip install -r requirements.txt
```

Listado 6.2. Instalación en Linux

Para arrancar la aplicación:

```
1 source .venv/bin/activate
2 cd src
3 streamlit run app.py
```

Listado 6.3. Arranque en Linux

A.5 Primera ejecución

Al arrancar, Streamlit abre automáticamente el navegador en `http://localhost:8501`. La primera vez que se consulta una sesión, FastF1 descarga los datos correspondientes y los almacena en caché local; este proceso puede tardar desde unos segundos hasta varios minutos según la sesión y la conexión. Las consultas posteriores a la misma sesión son prácticamente instantáneas.

Apéndice B. Manual de Usuario

TALOS se organiza en torno a un flujo de navegación de tres niveles: el usuario parte de un **buscador de Gran Premio**, selecciona una **sesión** concreta y accede a un **hub de módulos** de análisis. Toda la interacción se realiza desde el navegador, sin necesidad de conocimientos de programación.

B.1 Buscador de Gran Premio

La pantalla inicial presenta un mapamundi interactivo con todos los circuitos del calendario del año seleccionado. El usuario puede cambiar de temporada mediante el selector de año situado en la cabecera. Al hacer clic sobre el marcador la aplicación accede a la pantalla de selección de sesión correspondiente. La figura 6.1 muestra esta pantalla inicial.

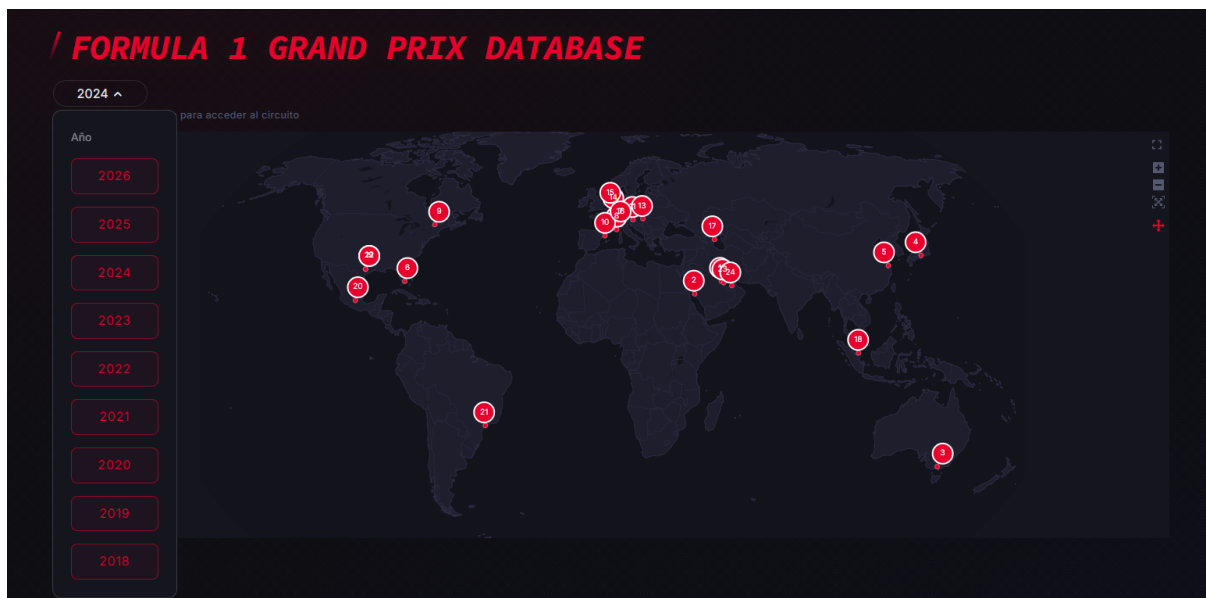


Figura 6.1. Buscador de Gran Premio: mapamundi interactivo del calendario.

B.2 Selección de sesión

Una vez seleccionado el Gran Premio, se muestran las sesiones disponibles y las sesiones de formato *Sprint* cuando proceda. Cada sesión se representa mediante una tarjeta, fecha y hora. Al pulsar *Cargar*, FastF1 descarga y sincroniza los datos de dicha sesión, como se muestra en la figura 6.2.

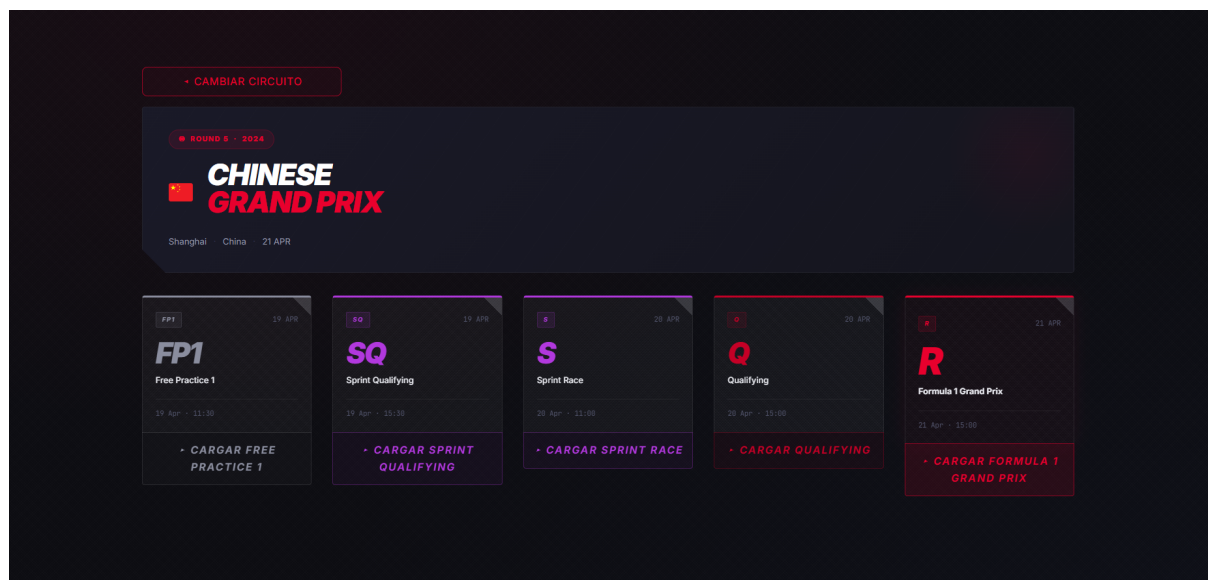


Figura 6.2. Pantalla de selección de sesión del fin de semana.

B.3 Hub de módulos

El hub es el punto central de la aplicación, muestra la sesión activa, un podio con los primeros clasificados y un panel lateral con métricas rápidas y la clasificación final. Los módulos de análisis se agrupan en cuatro categorías accesibles mediante pestañas. Desde cualquier módulo, el usuario puede volver al Hub. La figura 6.3 muestra este panel central.

B.4 Módulos de análisis

A continuación se describen los catorce módulos disponibles agrupados por categoría.

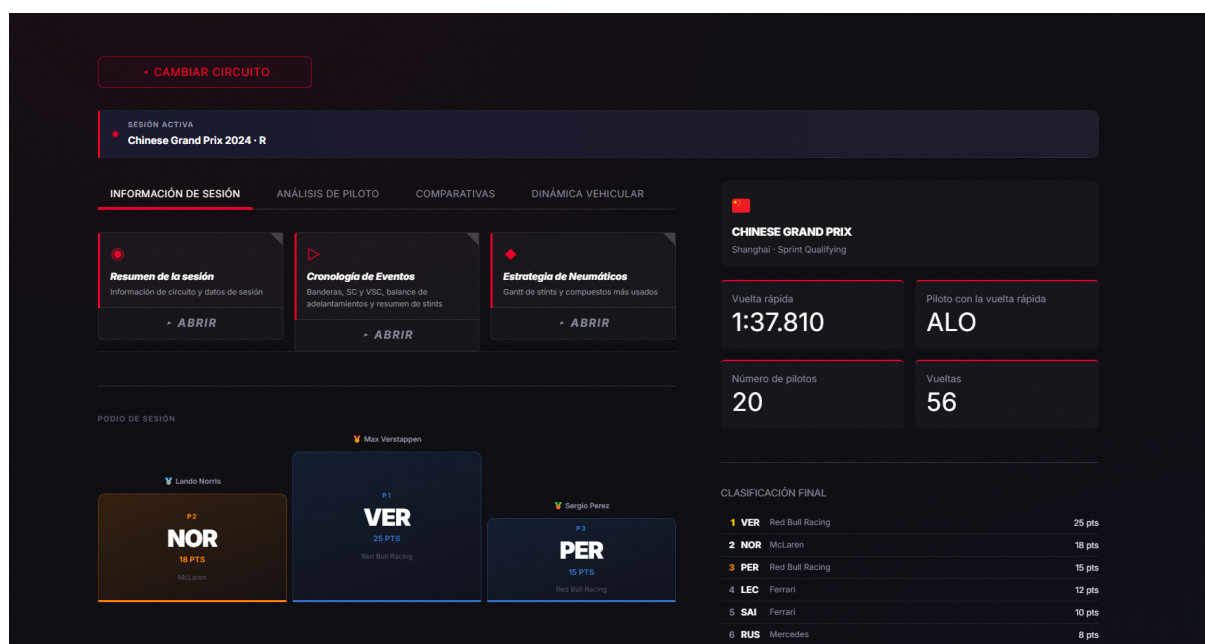


Figura 6.3. Hub de módulos con la sesión activa, el podio y el panel de métricas.

B.4.1 Información de Sesión

1. **Resumen de la sesión.** Información del circuito y datos generales de la sesión. Se muestran temperaturas relativas al resto de circuitos y datos generales del trazado. Se dibuja el trazado con los sectores coloreados, zonas de velocidad y DRS. Este módulo incluye el clasificador de circuito indicándonos en cuál de las 5 categorías se encuentra.
2. **Cronología de Eventos.** Reconstruye el transcurso de la sesión incluyendo las banderas, periodos de *Safety Car* y *Virtual Safety Car*, balance de adelantamientos que compara la posición inicial con la final y un resumen de los *stints* de cada piloto.
3. **Estrategia de Neumáticos.** Muestra, mediante un diagrama de Gantt, los *compuestos* usados por cada piloto y las gomas más utilizadas en toda la carrera, permitiendo comparar las estrategias seguidas a lo largo de la carrera.

B.4.2 Análisis de Piloto

1. **Resumen del Piloto.** Sintetiza la sesión de un piloto concreto indicando claves de rendimiento, tiempos por sector, y KPIs. Sirve como una primera introducción a lo que ha hecho el corredor elegido en la sesión.

2. **Mapa de Velocidad.** Trazado del circuito coloreado según la velocidad (km/h) a lo largo de la vuelta seleccionada. Evidencia las zonas de máxima y mínima velocidad.
3. **Mapa de Marchas.** Trazado del circuito coloreado según la marcha engranada en cada punto. Esto permite identificar los puntos de cambio y la gestión de caja, así como visualizar en qué marcha toma el piloto cada curva.
4. **Telemetry Trace.** Muestra los canales de telemetría del piloto junto con una animación del circuito y una caracterización de su estilo de conducción y tipo de vuelta. Muestra velocidad en todos los tramos del circuito, acelerador y freno, si ha activado o no el DRS, las RPM y marchas.

B.4.3 Comparativas

1. **Panel de Equipo.** Enfrenta a los dos pilotos de un mismo equipo, comparando su telemetría y ritmo para evaluar el rendimiento relativo entre compañeros. Incluye un trazado del circuito donde se puede apreciar el punto exacto del GPS de cada piloto para la vuelta dada. Como se muestra en la figura [6.4](#)

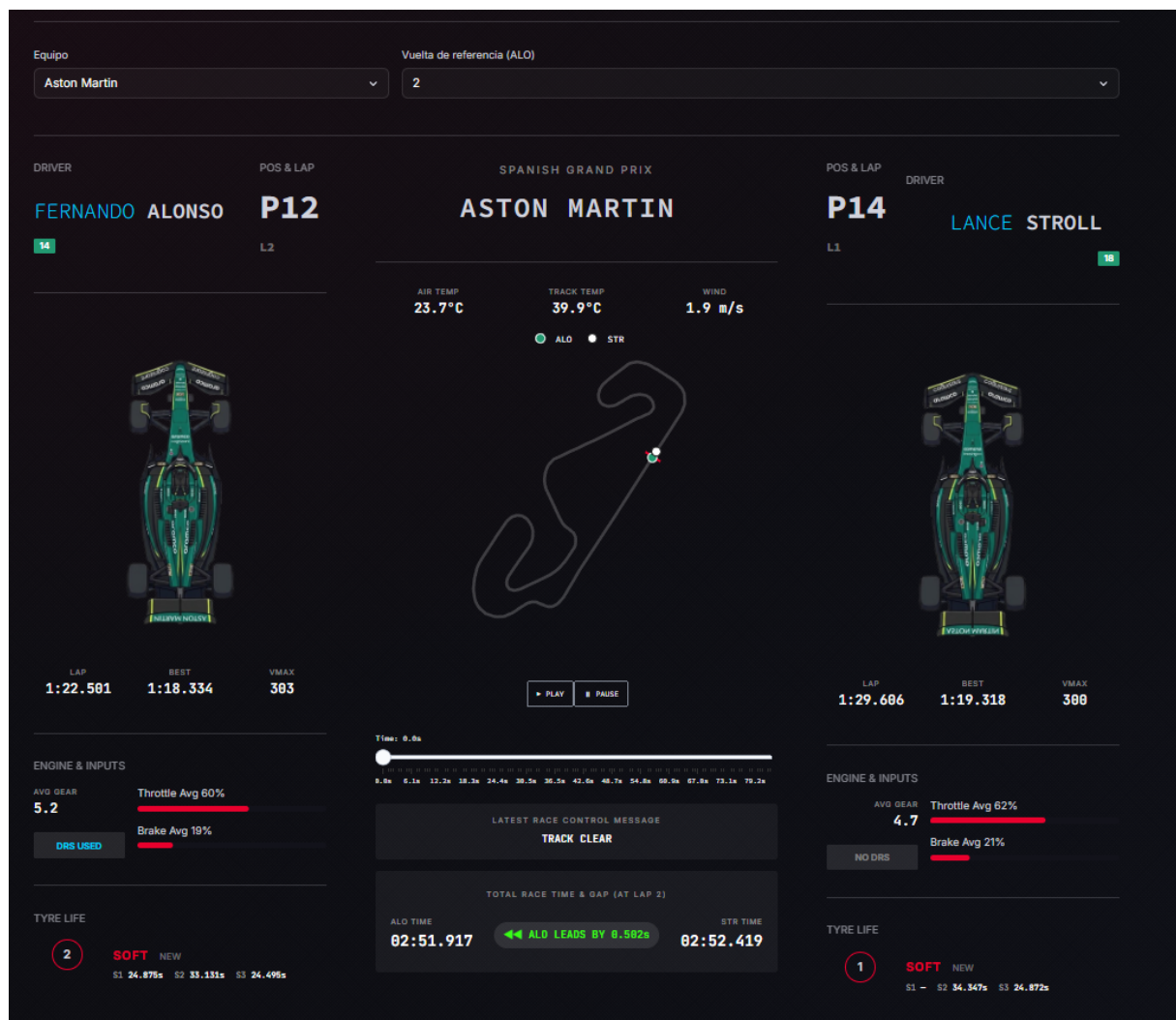


Figura 6.4. Módulo Panel de Equipo: comparación lado a lado de los dos pilotos de una escudería (telemetría, ritmo, neumáticos y circuito animado).

- Comparativa de Sesión.** Superpone (overlay) los canales de telemetría de varios pilotos sobre la misma vuelta, facilitando la comparación directa de sus métricas. Incluye una sección de dominancia por circuito en la que se pinta del color del piloto dominante el microsector correspondiente.
- La Vuelta Ideal.** Reconstruye la vuelta óptima a partir de los mejores microsectores de toda la parrilla, con posibilidad de filtrar por equipos, mostrando el límite teórico de rendimiento.
- Coche Fantasma.** Genera una animación 2D de persecución que enfrenta las vueltas de dos pilotos sobre el trazado, visualizando dónde y cuánto gana tiempo cada uno.

B.4.4 Dinámica Vehicular

1. **Diagrama G-G.** Representa el círculo de tracción y la envolvente de fuerzas G (longitudinales y laterales) del monoplace, mostrando el aprovechamiento del agarre disponible.
2. **Carga Aerodinámica.** Ofrece una estimación teórica de la carga aerodinámica a partir de la relación entre la aceleración lateral y la velocidad del coche.
3. **Mapa de Carga por Rueda.** Estima la distribución de carga por eje y la temperatura de frenos a lo largo de la vuelta, ilustrando las transferencias de peso en frenadas y curvas.

B.5 Selección de piloto y vuelta

Los módulos centrados en un piloto concreto incorporan dos selectores comunes: uno para elegir el piloto y otro para la vuelta a analizar. Por defecto se selecciona la vuelta más rápida del piloto. Adicionalmente se incluye la opción de mostrar curvas que superpone la numeración de las curvas sobre las visualizaciones que lo permiten.

Bibliografía

- [1] Felipe Agud and Charles Thraves. "Optimizing pit stop strategies in Formula 1 with dynamic programming and game theory". In: *European Journal of Operational Research* 319.3 (2024), pp. 908–919. DOI: [10.1016/j.ejor.2024.07.011](https://doi.org/10.1016/j.ejor.2024.07.011).
- [2] Amazon Web Services. *AWS Formula 1 Insights*. Socio oficial de nube e inteligencia artificial de la Fórmula 1 desde 2018. 2018. URL: <https://aws.amazon.com/sports/f1/>.
- [3] Paolo Aversa, Laure Cabantous, and Stefan Haeffliger. "When decision support systems fail: Insights for strategic information systems from Formula 1". In: *Journal of Strategic Information Systems* 27.3 (2018), pp. 221–236. DOI: [10.1016/j.jsis.2018.03.002](https://doi.org/10.1016/j.jsis.2018.03.002).
- [4] James Bekker and William Lotz. "Planning Formula One race strategies using discrete-event simulation". In: *Journal of the Operational Research Society* 60.7 (2009), pp. 952–961. DOI: [10.1057/palgrave.jors.2602626](https://doi.org/10.1057/palgrave.jors.2602626).
- [5] Miguel Cid Espeso. "Visualización de datos de Fórmula 1: diseño de un dashboard para seguimiento de carreras". MA thesis. Universidad Politécnica de Madrid, 2023. URL: <https://oa.upm.es/83177/>.
- [6] F1 Briefing. *Fuel Load vs. Lap Time: F1 Performance Analysis*. Efecto de la carga de combustible sobre el tiempo por vuelta (aprox. 0,03 s/kg). 2024. URL: <https://f1briefing.com/fuel-load-vs-lap-time-f1-performance-analysis/>.
- [7] F1-Tempo. *F1-Tempo: Formula 1 telemetry comparison tool*. Construido con React y TypeScript sobre FastF1. 2023. URL: <https://www.f1-tempo.com>.
- [8] Formula 1. *Watch: Russell dramatically crashes out while chasing Alonso on final lap in Australia*. Fórmula 1, 24 de marzo de 2024. 2024. URL: <https://www.formula1.com/en/latest/article/watch-russell-dramatically-crashes-out-while-chasing-alonso-on-final-lap-in.3LEuxMY2ykTHVjPrqLmkLu>.
- [9] Thomas D. Gillespie. *Fundamentals of Vehicle Dynamics*. Warrendale, PA: SAE International, 1992. DOI: [10.4271/R-114](https://doi.org/10.4271/R-114).

- [10] Alex Guerrero and Robert Castilla. "Aerodynamic Study of the Wake Effects on a Formula 1 Car". In: *Energies* 13.19 (2020), p. 5183. DOI: [10.3390/en13195183](https://doi.org/10.3390/en13195183).
- [11] Charles R. Harris et al. "Array Programming with NumPy". In: *Nature* 585 (2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [12] Alexander Heilmeyer et al. "Virtual Strategy Engineer: Using Artificial Neural Networks for Making Race Strategy Decisions in Circuit Motorsport". In: *Applied Sciences* 10.21 (2020), p. 7805. DOI: [10.3390/app10217805](https://doi.org/10.3390/app10217805).
- [13] Jolpica contributors. *Jolpica-F1: Ergast-compatible Formula 1 REST API*. Sustituto oficial de Ergast API tras su deprecación en 2024. 2024. URL: <https://github.com/jolpica/jolpica-f1>.
- [14] Joseph Katz. "Aerodynamics of Race Cars". In: *Annual Review of Fluid Mechanics* 38 (2006), pp. 27–63. DOI: [10.1146/annurev.fluid.38.050304.092016](https://doi.org/10.1146/annurev.fluid.38.050304.092016).
- [15] Arturo Latorre Castelltort. "F1 Data Analysis". MA thesis. Universitat de Barcelona, 2025. URL: <https://diposit.ub.edu/dspace/handle/2445/221250>.
- [16] Wes McKinney. "Data Structures for Statistical Computing in Python". In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
- [17] McLaren Applied. *ATLAS: Advanced Telemetry Linked Acquisition System*. 2023. URL: <https://www.motionapplied.com/products/ATLAS>.
- [18] William F. Milliken and Douglas L. Milliken. *Race Car Vehicle Dynamics*. Warrendale, PA: SAE International, 1995.
- [19] David Morán Montero. "Análisis de datos de telemetría de Fórmula 1 con técnicas de Deep Learning". MA thesis. Universidad de Extremadura, 2022. URL: <https://dehesa.unex.es/handle/10662/15838>.
- [20] Chris Newell. *Ergast Motor Racing Developer API*. API deprecada en diciembre de 2024. Cobertura histórica desde la temporada 1950. 2005. URL: <https://web.archive.org/web/20230607021710/https://ergast.com/mrd/>.
- [21] Oehrly and contributors. *FastF1: A Python package for accessing and analyzing Formula 1 data*. Version 3.8.2. 2020. URL: <https://github.com/theOehrly/Fast-F1>.
- [22] OpenF1 contributors. *OpenF1: Free and open-source Formula 1 API*. 2023. URL: <https://openf1.org>.

- [23] Hans B. Pacejka. *Tire and Vehicle Dynamics*. 3rd ed. Oxford: Butterworth-Heinemann (Elsevier), 2012.
- [24] Paolo Patierno. *Real-time monitoring of Formula 1 telemetry data on Kubernetes with Grafana, Apache Kafka and Strimzi*. Grafana Labs blog. 2021. URL: <https://grafana.com/blog/2021/02/02/real-time-monitoring-of-formula-1-telemetry-data-on-kubernetes-with-grafana-apache-kafka-and-strimzi/>.
- [25] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. URL: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [26] Plotly Technologies Inc. *Collaborative Data Science*. 2015. URL: <https://plot.ly>.
- [27] Abraham Savitzky and Marcel J. E. Golay. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures". In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639. DOI: [10.1021/ac60214a047](https://doi.org/10.1021/ac60214a047).
- [28] Ronald W. Schafer. "What Is a Savitzky-Golay Filter?" In: *IEEE Signal Processing Magazine* 28.4 (2011), pp. 111–117. DOI: [10.1109/MSP.2011.941097](https://doi.org/10.1109/MSP.2011.941097).
- [29] Streamlit Inc. *Streamlit: A Faster Way to Build and Share Data Apps*. 2019. URL: <https://streamlit.io>.
- [30] Fraser Tarbet. *F1Dash: Interactive Formula 1 analytical dashboard*. 2022. URL: <https://github.com/FraserTarbet/F1Dash>.
- [31] Devin Thomas et al. "Explainable Reinforcement Learning for Formula One Race Strategy". In: *Proceedings of the ACM Symposium on Applied Computing (SAC)*. arXiv:2501.04068. 2025. URL: <https://arxiv.org/abs/2501.04068>.
- [32] Pauli Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [33] Alexander Wischnewski et al. "Vehicle Dynamics State Estimation and Localization for High Performance Race Cars". In: *IFAC-PapersOnLine (10th IFAC Symposium on Intelligent Autonomous Vehicles)*. Vol. 52. 8. 2019, pp. 307–312. DOI: [10.1016/j.ifacol.2019.08.064](https://doi.org/10.1016/j.ifacol.2019.08.064).



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga